F25-079-D-OratoAI

Project Team

Muhammad Shayan Memon	22i-0773
Ali Haider	22I-1210
Awais Khan	22I-0997

Session 2022-2026

Supervised by

Ms. Saira Qamar

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2025

Contents

List of Figures

List of Tables

Chapter 1

Introduction

OratoAI is an AI-powered oral presentation analysis tool designed specifically for Pakistani students, aiming to provide objective, automated, and culturally localized feedback to help improve their presentation skills. The project addresses widespread challenges faced by students in receiving consistent, actionable guidance on vocal clarity, body language, and content relevance, particularly due to a lack of tools that accommodate local accent and presentation style. By analyzing pre-recorded solo presentation videos and generating tailored reports reflecting Pakistani English accent and presentation norms, OratoAI seeks to bridge the gap in accessible, scalable, and quality coaching.

1.1 Existing Solutions & Their Limitations

Table 1.1: Comparison of Existing Solutions and Their Limitations

Product	**Feedback Quality**	**Pakistani Adaptation**	**Real-Time Video Support**	**Open Source Resources**
Yoodli	Good	No	Yes	No
Orai	Moderate	No	No	No
VirtualSpeech	Good	No	Yes (Paid)	No
OratoAI (Plan)	High	Yes	Yes	Yes

1.1.1 Analysis of Existing Solutions

1.1.1.1 Yoodli

Yoodli is an AI-powered speech coaching platform that provides users with feedback on their presentation and communication skills. The system offers good quality feedback

focusing on speech patterns, filler words, and pacing. It supports real-time video analysis, allowing users to practice and receive immediate feedback.

Limitations:

- No adaptation for Pakistani accents or cultural context
- Lacks open-source resources for customization
- Limited accessibility for local educational institutions

1.1.1.2 Orai

Orai is a mobile-based public speaking coach that analyzes speech delivery and provides feedback on various aspects of oral presentations. The platform offers moderate quality feedback with a focus on basic speech metrics.

Limitations:

- Does not support real-time video analysis
- No Pakistani accent adaptation
- Closed-source platform with no customization options
- Limited comprehensive feedback on body language and gestures

1.1.1.3 VirtualSpeech

VirtualSpeech is a virtual reality and web-based training platform for improving public speaking and presentation skills. It provides good quality feedback with immersive VR experiences and supports real-time video analysis through paid subscriptions.

Limitations:

- Video support only available in paid versions
- No adaptation for Pakistani accents or regional speech patterns
- Proprietary system with no open-source components
- High cost barrier for widespread educational adoption

1.1.1.4 OratoAI (Proposed Solution)

OratoAI is designed to address the limitations identified in existing solutions by providing:

- High-quality feedback tailored to Pakistani students
- Accent-aware speech recognition optimized for Pakistani English
- Video support with comprehensive body language analysis
- Open-source resources enabling customization and widespread adoption
- Cultural localization for Pakistani educational context
- Cost-effective solution accessible to educational institutions

1.2 Problem Statement

Public speaking and presentation skills are critical for academic and professional success. Many Pakistani students receive limited, subjective, or infrequent feedback on their oral presentations due to instructor workload and resource constraints. Off-the-shelf international tools often underperform for local users because they do not account for Pakistani English accents and regional presentation styles, resulting in lower transcription accuracy and less useful feedback. Educators cannot consistently provide individualized, iterative coaching at scale. Consequently, students have fewer practice cycles and less actionable guidance to improve their delivery before graded assessments or interviews. OratoAI addresses this gap by offering an accessible, automated, and culturally-aware analysis of recorded presentations to help students self-improve and help teachers scale assessment.

1.3 Scope

OratoAI is a web based application limited to analyzing pre-recorded solo presentation videos. The scope includes:

- Upload of common video formats (MP4) with client-side validation and configurable size limits.
- Server-side preprocessing to extract audio and frames for downstream ML tasks.
- Computer-vision-based posture and gesture analysis (pose keypoints, head orientation, gesture counts).

- Accent-aware speech recognition (timestamped transcripts, filler-word detection, speech-rate metrics) tuned or evaluated for Pakistani English.
- LLM-assisted content relevance checking against a user-declared topic, with optional web-scraped evidence for simple factual checks.
- Aggregation of module outputs into explainable sub-scores accessible via a dashboard.

1.4 Modules

The system is organized into modular components to ease development and testing:

- Video Upload & Preprocessing
- Computer Vision: Pose & Gesture Analysis
- Accent-Aware ASR & Speech Metrics
- Content Relevance & Verification (LLM-assisted)
- Aggregator: Scoring & Feedback Generation
- Dashboard & Reporting (Student views)

1.5 Work Division

Table 1.2: Work Division Table

Name	**Registration No.**	**Responsibilities / Modules / Features**
M. Awais Khan	221-0997	**Module:** Accent-Aware Speech Recognition • Metrics Calculation • ASR evaluation • Integration with CV Module
Ali Haider	22i-1210	**Module:** Computer Vision – Pose and Gesture Analysis • Develop posture detection and gesture metrics • Content Relevance & Verification (LLM-assisted) • Integration with ASR Module for Content Relevance
Shayan Memon	22i-0773	**Module:** Backend & Frontend • Backend orchestration (FastAPI/Flask) • Web scraping and LLM content relevancy • UI design, upload, feedback dashboard

Chapter 2

Project Requirements

This chapter comprehensively details the functional and non-functional requirements of the OratoAI project, outlining essential expectations for system behavior, performance, and quality. It presents use-case and event–response diagrams that depict the main user interactions and system responses, supporting deeper understanding of core workflows. Further, module-level functional requirements are specified for each component including video upload, speech and pose analysis, report generation, and dashboard functionality ensuring clarity of deliverables and facilitating robust testing. To guarantee a high standard of experience, quality targets such as response time, accuracy of feedback, usability metrics, and privacy benchmarks are defined, establishing measurable goals for the platform.

2.1 Use-case / Event-Response Table / Storyboarding

2.1.1 Use Case Diagram

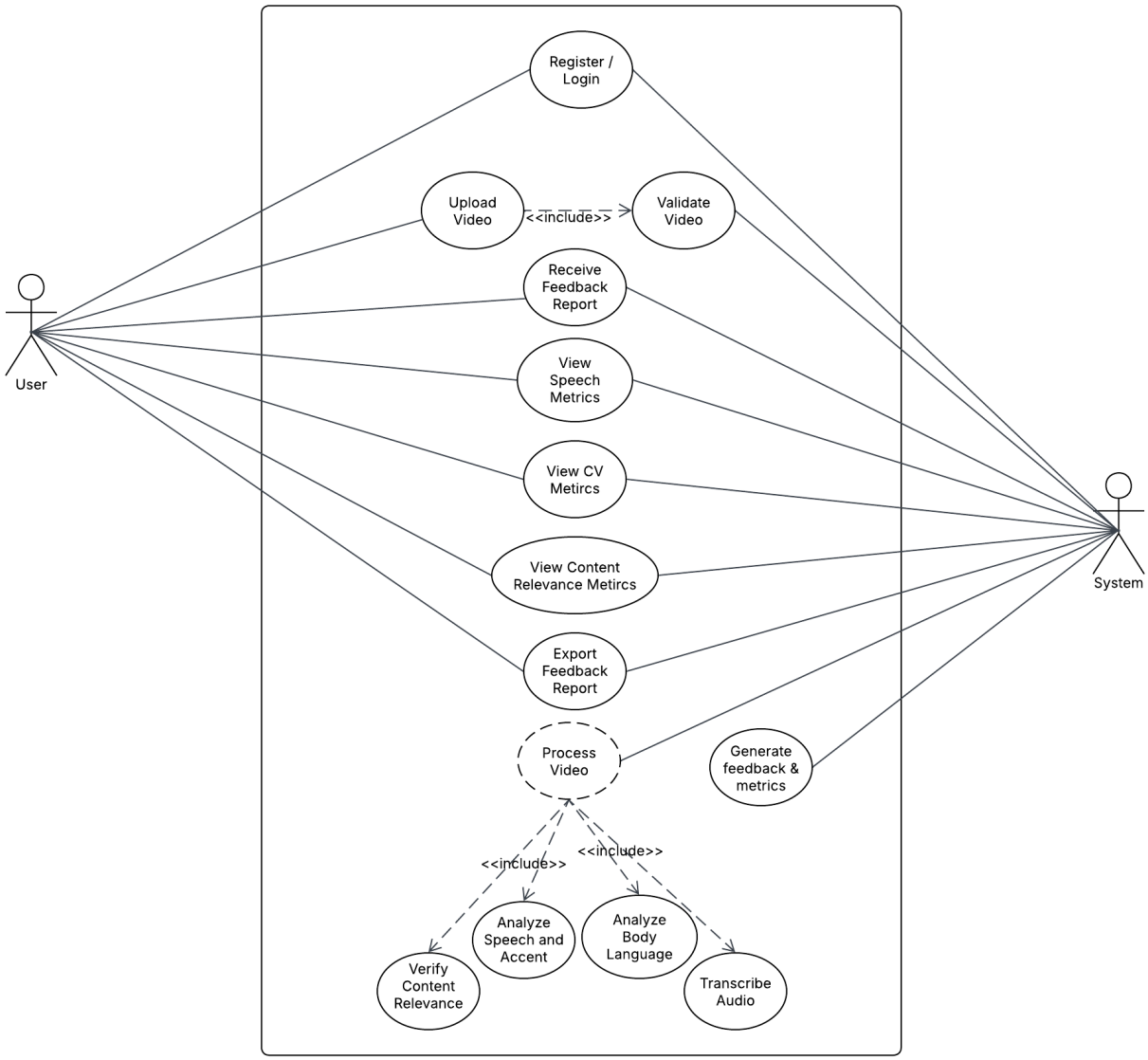

Figure 2.1: OratiAI Use case Diagram

2.1.2 Primary Use Cases

The following table summarizes the main use-cases for OratoAI. (Actors: User, System)

Actor	Use Case	Description
User	User Registration	User registers a new account or logs into the system to access their dashboard.
User	Upload and Validate Video	User uploads a presentation video, which the system validates for format and compatibility before processing.
System	Preprocess Video	System automatically extracts audio and video frames from the uploaded submission to prepare for analysis.
System	Analyze Pose and Gestures	System analyzes video frames to evaluate metrics related to posture, body language, and hand gestures.
System	Analyze Speech and Accent	System transcribes audio using an accent-aware model and analyzes vocal metrics like speech rate, pauses, and filler words.
System	Verify Content Relevance	System uses an LLM to assess how well the spoken content aligns with the presentation's stated topic.
User	Receive Feedback Report	User receives a comprehensive report aggregating all analysis results from vocal, visual, and content modules.
User	Access Dashboard	User interacts with a personal dashboard to view historical submissions and track performance progress over time.
User	Export Report	User downloads the detailed feedback report in PDF or CSV format for offline reference or record-keeping.
User	Data Management	User requests the permanent deletion,updation of their account and all associated data from the platform.

Table 2.1: Primary use-cases for OratoAI

2.1.3 Fully Dressed Use Cases

2.1.3.1 UC-01: User Registration and Login

Actor(s): User

Description: A new or returning user securely accesses their account on the OratoAI platform.

Preconditions: The user has access to the OratoAI web application.

Postconditions: The user is successfully authenticated, a session is created, and they are directed to their personal dashboard.

Main Success Scenario:

1. The user enters their credentials.
2. The system validates the credentials.
3. The user is authenticated and a session is created.
4. The user is redirected to their dashboard.

Extensions: **Invalid Credentials:** The system shows an error message.

2.1.3.2 UC-02: Upload and Validate Video

Actor(s): User

Description: The user uploads a presentation video for analysis.

Preconditions: The user is logged in.

Postconditions: A valid video is stored, and a processing job is queued.

Main Success Scenario:

1. The user selects an MP4 file and provides a presentation topic.
2. The system validates the file format and size.
3. The video is stored, and the system confirms the upload.

Extensions: **Invalid File:** The system rejects the upload and displays an error.

2.1.3.3 UC-03: Preprocess Video

Actor(s): System

Description: The system extracts audio and video frames for analysis.

Preconditions: A valid video has been uploaded.

Postconditions: Audio and frame artifacts are stored and ready for analysis.

Main Success Scenario:

1. The system initiates the preprocessing job.
2. It extracts the audio stream and video frames.
3. All artifacts are saved to storage.
4. The job status is updated.

Extensions: **Corrupted Data:** The job status is set to "Failed," and the user is notified.

2.1.3.4 UC-04: Analyze Pose and Gestures

Actor(s): System

Description: The system analyzes video frames for body language metrics.

Preconditions: Video frames have been successfully extracted.

Postconditions: Posture and gesture metrics are calculated and stored.

Main Success Scenario:

1. The `PoseEstimator` analyzes frames for keypoints.
2. `PostureMetrics` calculates metrics like slouch duration.
3. The `GestureAnalyzer` detects hand gestures.
4. All data is saved and linked to the video submission.

Extensions: **Low-Quality Video:** The system flags the module as incomplete and logs a warning.

2.1.3.5 UC-05: Analyze Speech and Accent

Actor(s): System

Description: The system transcribes speech and calculates vocal metrics.

Preconditions: The audio has been successfully extracted.

Postconditions: A timestamped transcript and speech metrics are stored.

Main Success Scenario:

1. The `ASRModule` transcribes the audio using an accent-aware model.
2. The `SpeechMetricsAnalyzer` computes metrics (e.g., filler words, speech rate).
3. The transcript and all metrics are stored.

Extensions: **Unrecognized Speech:** The system flags segments with low confidence scores for user review.

2.1.3.6 UC-06: Verify Content Relevance

Actor(s): System

Description: An LLM assesses the alignment of the speech with the declared topic.

Preconditions: The transcription is complete and a topic was provided.

Postconditions: A content relevance score is generated.

Main Success Scenario:

1. The `ContentVerifier` is triggered with the transcript and topic.
2. The `LLMModule` generates a relevance score.
3. The transcript is annotated with the score and highlights.

Extensions: **LLM Fails:** The system logs an error and skips this section in the report.

2.1.3.7 UC-07: Receive Feedback Report

Actor(s): User

Description: The user views the aggregated report of all analysis results.

Preconditions: All analysis modules have completed.

Postconditions: The user can view a detailed, interactive report.

Main Success Scenario:

1. The user opens the report from their dashboard.
2. The `ReportAggregator` gathers results from all modules.
3. The system displays the final report with scores, visuals, and tips.

Extensions: **Data Missing:** An error message is displayed to the user.

2.1.3.8 UC-08: Access Dashboard

Actor(s): User

Description: The user views past submissions and tracks progress.

Preconditions: The user is logged in.

Postconditions: The user has reviewed their historical performance.

Main Success Scenario:

1. The user navigates to the dashboard.
2. The system retrieves all past submissions and reports.
3. Performance trends and a list of reports are displayed.
4. The user can click any report to view its details.

Extensions: **No Submissions:** A welcome message is displayed with a call-to-action.

2.1.3.9 UC-09: Export Report

Actor(s): User

Description: The user downloads a feedback report in PDF or CSV format.

Preconditions: The user is viewing a generated report.

Postconditions: A file is downloaded to the user's device.

Main Success Scenario:

1. The user clicks "Export as PDF" or "Export as CSV".
2. The `ExportService` generates the file.
3. The user's browser prompts them to save the file.

Extensions: **Export Fails:** An error message is displayed.

2.1.3.10 UC-10: Data Management

Actor(s): User

Description: The user requests the permanent deletion/updation of their account and data.

Preconditions: The user is logged into their account.

Postconditions: All user-associated data is managed properly.

Main Success Scenario:

1. The user initiates a data management request.
2. The `UpdationService` deletes all submissions and artifacts.
3. The user's account information is deleted, and their session is terminated.

Extensions: **Deletion Fails:** The action is rolled back, and the user is instructed to contact support.

2.1.4 Event–Response Table

Event (Stimulus)	System Response
User uploads presentation video	Validate file type and size; store media; confirm or reject upload; show error message if invalid.
Video preprocessing initiated	Extract frames and audio; normalize audio; flag any corrupted or incomplete data; update user on progress.
Pose/gesture analysis triggered	Run computer vision algorithms; compute keypoints and gestures; report if input video quality is too low.
Accent-aware ASR activated	Transcribe speech using customized ASR models; log confidence scores.
Filler word and speech rate analysis requested	Detect filler words, pauses, and rate; insert markers in transcript; return clean summary in feedback report.
Content relevance check started	Compare transcript with declared topic using LLM/web data; highlight off-topic statements; notify if analysis fails.
Feedback report generation	Aggregate module outputs; produce summary and suggestions; notify user when report is ready.
User accesses dashboard/report	Fetch feedback records securely; display analytics and improvement tips, or error message if missing.
User requests download/export of report	Generate downloadable PDF or CSV; verify privacy and permissions; handle errors gracefully.
System error or crash detected	Log error; present user with recovery message and safe retry or contact option.

Table 2.2: System responses to key events and stimuli in OratoAI.

2.1.5 Storyboarding (Main user flow)

1. **Login / Landing**: User logs in and arrives at the dashboard showing previous submissions, summarized performance trends, and a prominent *Upload* button.

2. **Upload Screen**: User drags/drops or selects a presentation video file, declares the presentation topic, and clicks *Submit*. Client interface validates file type and size before upload.

3. **Processing Status**: The system shows status updates (Queued $\rightarrow$ Processing $\rightarrow$ Done), displays estimated time to completion.

4. **Feedback Report**: Once analysis completes, the report screen presents overall feedback scores, module sub-scores (Speech, Body Language, Content Relevance), a timeline with clickable timestamps, sample annotated video frames, and specific

improvement tips. Action buttons: Download PDF, Download CSV, Re-submit for another attempt.

5. **Dashboard / Analytics**: User reviews historical submissions, tracks longitudinal progress via dashboard visuals, and compares personal performance across multiple presentations over time.

6. **Privacy / Settings**: User accesses privacy controls, initiates deletion of uploaded content, manages retention period preferences, and updates account settings.

7. **Logout**: User exits the platform securely, ensuring all session data is cleared.

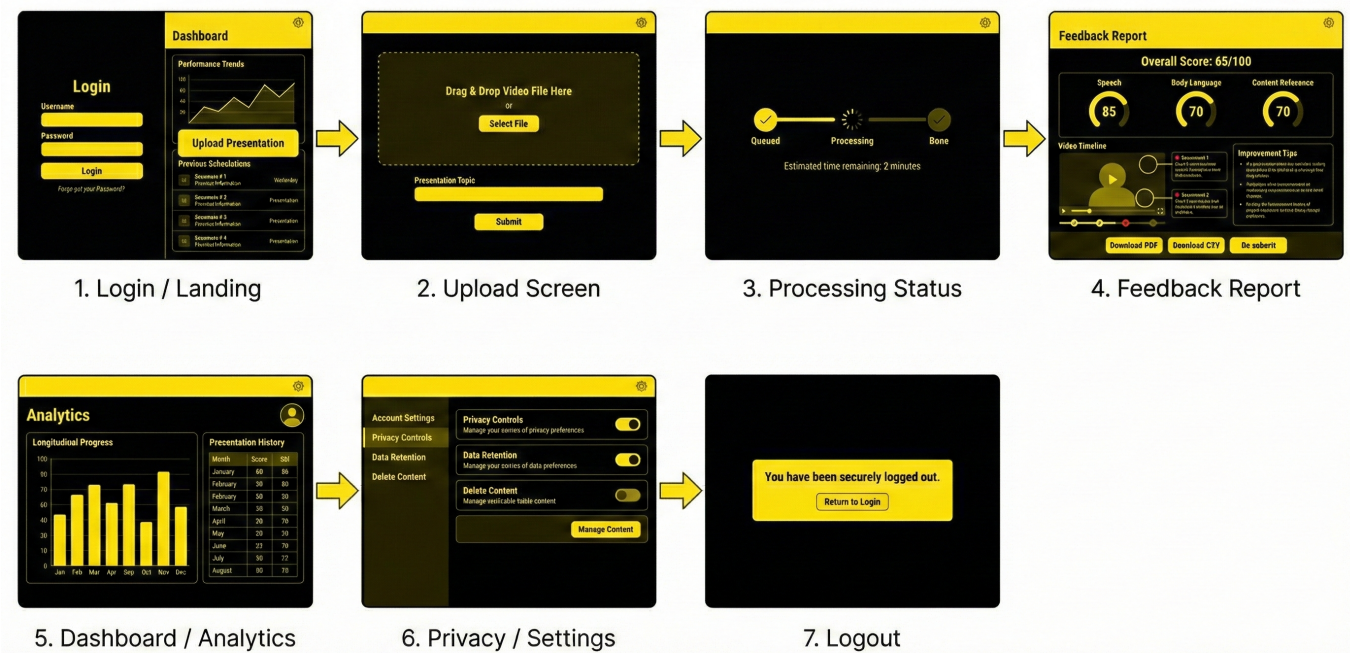

Figure 2.2: Storyboarding

2.2 Functional Requirements

This section describes the functional requirements organized by module. Each requirement is stated in natural language and mapped to FR numbers used elsewhere in project documentation.

2.2.1 Module: User Management & Auth

FR1. **FR1: Registration & Login (Must)**: The system SHALL allow users to create accounts and log in to the application using secure credentials.

FR2. **FR2: Role-based Access (Must)**: The system SHALL enforce role-based access control so that users can only see their own submissions and feedback.

FR3. **FR3: Video Management (Must)**: The system SHALL allow users to manage consent and control deletion of their personal data and submitted videos.

2.2.2 Module: Video Upload & Preprocessing

FR4. **FR4: Manage Upload (Must)**: The system SHALL accept MP4 video files only, with a configurable maximum file size, providing clear error messages for invalid inputs.

FR5. **FR5: Preprocessing (Must)**: Upon upload, the system SHALL extract video frames at a configurable rate and normalize audio for further analysis.

2.2.3 Module: Computer Vision — Pose & Gesture Analysis

FR6. **FR6: Pose Keypoint Extraction (Must)**: The system SHALL perform pose estimation on each video frame, capturing keypoints and facial landmarks.

FR7. **FR7: Gesture and Posture Metrics (Must)**: The system SHALL calculate body language metrics such as slouch duration, hand gestures, and head orientation with corresponding example screenshots.

FR8. **FR8: Motion Analysis (Should)**: The system SHOULD analyze gesture velocity and movement consistency as additional feedback metrics.

2.2.4 Module: Accent-Aware ASR & Speech Metrics

FR9. **FR9: Speech Transcription (Must)**: The system SHALL generate accurate timestamped transcripts optimized for Pakistani English accents.

FR10. **FR10: Speech Quality Metrics (Must)**: The system SHALL detect filler words, calculate speech rate, and measure pause durations, including timestamps for each event.

FR11. **FR11: Fluency and Articulation Assessment (Should)**: The system SHOULD assess articulation clarity and fluency level as part of speech evaluation.

2.2.5 Module: Content Relevance & Verification

FR12. **FR12: Topic Matching (Must)**: The system SHALL compare the transcript with the user-declared topic to generate a relevance score and highlight off-topic segments.

FR13. **FR13: Factual Validation (Could)**: The system MAY flag factual inaccuracies and provide evidence snippets based on web-scraped data or language model analysis.

2.2.6 Module: Aggregation & Feedback Generation

FR14. **FR14: Composite Scoring (Must)**: The system SHALL combine individual module scores into a comprehensive feedback score with transparent weighting.

FR15. **FR15: Feedback Report Generation (Must)**: The system SHALL create detailed, actionable feedback reports with text, visuals, and example video frames.

FR16. **FR16: Export Options (Should)**: The system SHOULD offer PDF and CSV formats for report downloads.

2.3 Non-Functional Requirements

Non-functional requirements are quality attributes that the OratoAI system must satisfy. They are specific, measurable, and testable wherever possible to ensure a robust and user-friendly platform.

2.3.1 Reliability

- **NFR1 (Monitoring and Alerts):** Operational metrics including job queues, error rates, and performance latency SHALL be collected and trigger alerts upon threshold breaches.

2.3.2 Usability

- **NFR2 (Efficiency):** Users SHALL be able to upload a video and access feedback reports within **5 minutes** on a 20 Mbps internet connection.
- **NFR3 (Error Handling):** The system SHALL present concise, informative error messages for common issues like file size violations, unsupported formats, or processing failures.

2.3.3 Performance

- **NFR4 (Processing Latency):** Complete processing of a typical 2-minute presentation video, including preprocessing, ASR, CV, and content validation SHALL finish within **3 minutes** on standard GPU-enabled infrastructure.
- **NFR5 (API Responsiveness): 95%** of API calls excluding heavy processing jobs SHALL respond within **500 ms**.
- **NFR6 (UI Responsiveness):** Dashboard pages SHALL fully render in under **2 seconds** on modern desktop browsers with stable internet.

2.3.4 Security

- **NFR7 (Data Confidentiality):** All user data including media, transcripts, and reports SHALL be encrypted at rest and in transit, accessible only to authorized users.
- **NFR8 (Data Integrity):** The system SHALL ensure data integrity through checksums and audit trails preventing unauthorized modifications or deletions.
- **NFR9 (Audit Logging):** All critical user and admin actions SHALL be logged with timestamps and identifiers for compliance and troubleshooting.

2.3.5 Maintainability & Extensibility

- **NFR10 (Modularity):** Core components such as ASR, computer vision, and content verification SHALL be modular to allow independent updates without full system redeployments.
- **NFR11 (Test Coverage):** Backend services SHALL achieve a minimum of **70%** unit and integration test coverage to ensure robustness during development and model updates.

Chapter 3

System Overview

OratoAI is an AI-powered web application designed to assist Pakistani students in improving their oral presentation skills through automated feedback. The system analyzes pre-recorded solo presentation videos by processing audio and video data to generate detailed reports focusing on vocal clarity, body language, and content relevance. OratoAI addresses the challenge faced by many students, who lack consistent and objective feedback tailored to their accent and presentation style, by providing a scalable, culturally localized, and data-driven coaching platform.

The system workflow begins with secure video upload, where students submit their presentation recordings along with the declared topic. The backend preprocesses the video to extract audio and frames, enabling multiple machine learning pipelines to function effectively. Computer vision models perform pose estimation and gesture analysis to examine body language components such as posture, gestures, and head orientation. Concurrently, accent-aware automatic speech recognition (ASR) models, transcribe the speech into text while also computing speech quality metrics like filler word frequency, speech rate, and pauses.

Subsequently, large language models (LLMs) assist in verifying the relevance of the spoken content against the declared topic, highlighting off-topic or inconsistent segments. Aggregating outputs from these modules, OratoAI generates comprehensive feedback reports with actionable recommendations, visual illustrations, and confidence scores. These reports are accessible to students through an intuitive dashboard that supports progress tracking and report downloads.

OratoAI's modular architecture ensures each component—user management, video processing, ASR, computer vision, content verification, and feedback generation—can be independently developed, tested, and updated. The system prioritizes data privacy and security by requiring explicit user consent, encrypting data, and allowing users to manage their data retention preferences. The platform is designed to operate efficiently on

GPU-enabled servers with fallback CPU processing, maintaining a balance between responsiveness and resource utilization.

By leveraging state-of-the-art AI technologies and tailored data models, OratoAI aims to democratize presentation coaching in Pakistan, empowering students to improve their communication skills autonomously and effectively.

3.1 Architectural Design

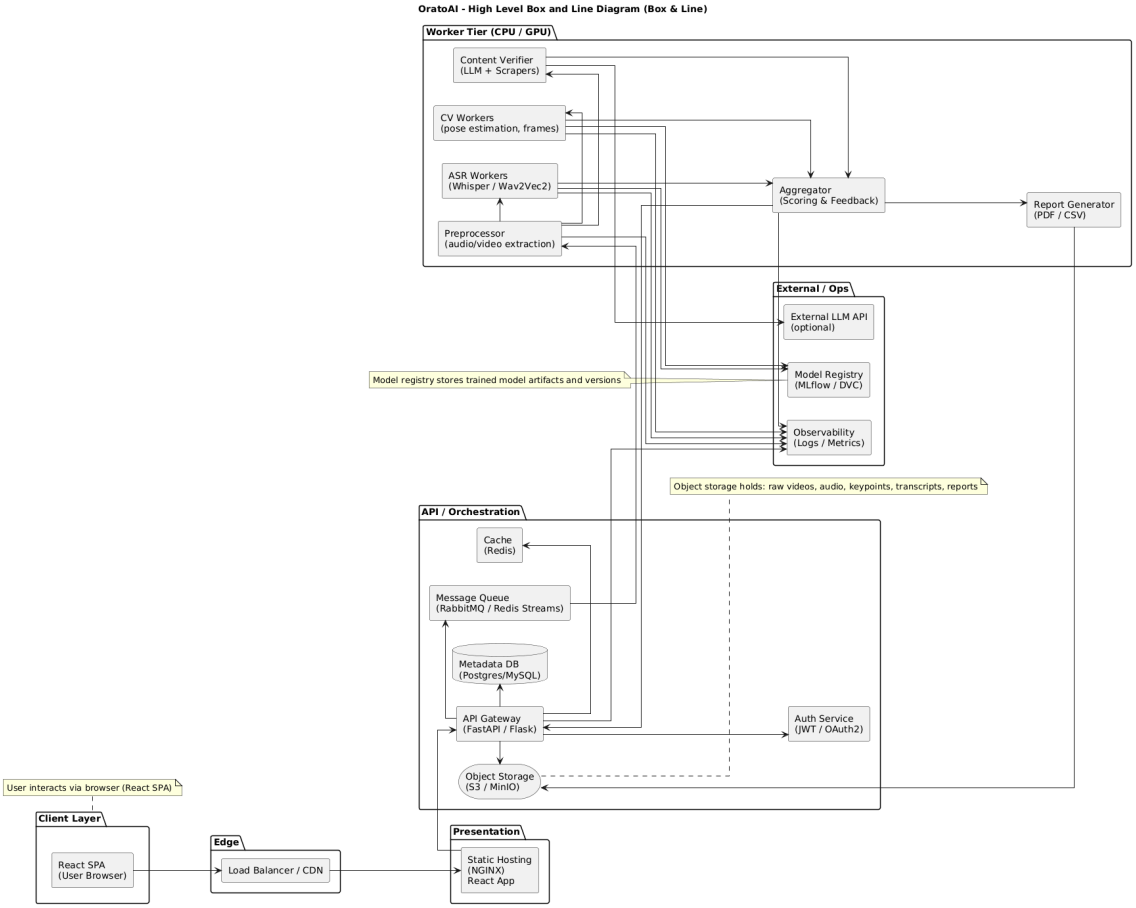

Figure 3.1: Architectural Design.

3.2 Data Design

OratoAI employs a modern, scalable architecture consisting of React frontend, Python backend with FastAPI, PostgreSQL database, and MinIO object storage for efficient data management and processing.

Key data entities and storage include:

- **Users:** Stored as relational tables in PostgreSQL with structured fields for authentication credentials, user profiles, privacy consents, and presentation submission history. Foreign key relationships ensure data integrity across user-related entities.
- **Presentation Video Metadata:** PostgreSQL tables store comprehensive video metadata including submission timestamps, file formats, sizes, user associations, declared topics, and processing job status. Actual media files are stored separately in MinIO object storage with secure access controls.
- **Multimedia Files:** Video files, extracted audio streams, and generated feedback reports (PDF/CSV) are stored in MinIO buckets, providing S3-compatible object storage with versioning and access management separate from the database.
- **Analysis Data and Metrics:** Extracted video frames, pose keypoints, speech transcripts, and content analysis results are stored in PostgreSQL using native JSON/JSONB columns for flexible querying of complex nested data structures while maintaining relational integrity.
- **Backend Services:** FastAPI server provides RESTful APIs leveraging SQLAlchemy ORM for database operations, handles asynchronous task scheduling for video processing, and interfaces with AI/ML pipelines. Redis integration supports caching and background job queue management.
- **Frontend Interaction:** The React frontend communicates with FastAPI endpoints to upload videos, download analysis reports, visualize feedback dashboards, and manage user settings, ensuring a responsive and interactive user experience.

This data design leverages PostgreSQL's ACID compliance and JSON support for structured and semi-structured data, MinIO's scalable object storage for multimedia files, and FastAPI's high-performance async capabilities, creating a robust foundation for AI-powered presentation analysis with seamless integration between all system components.

3.3 Domain Model

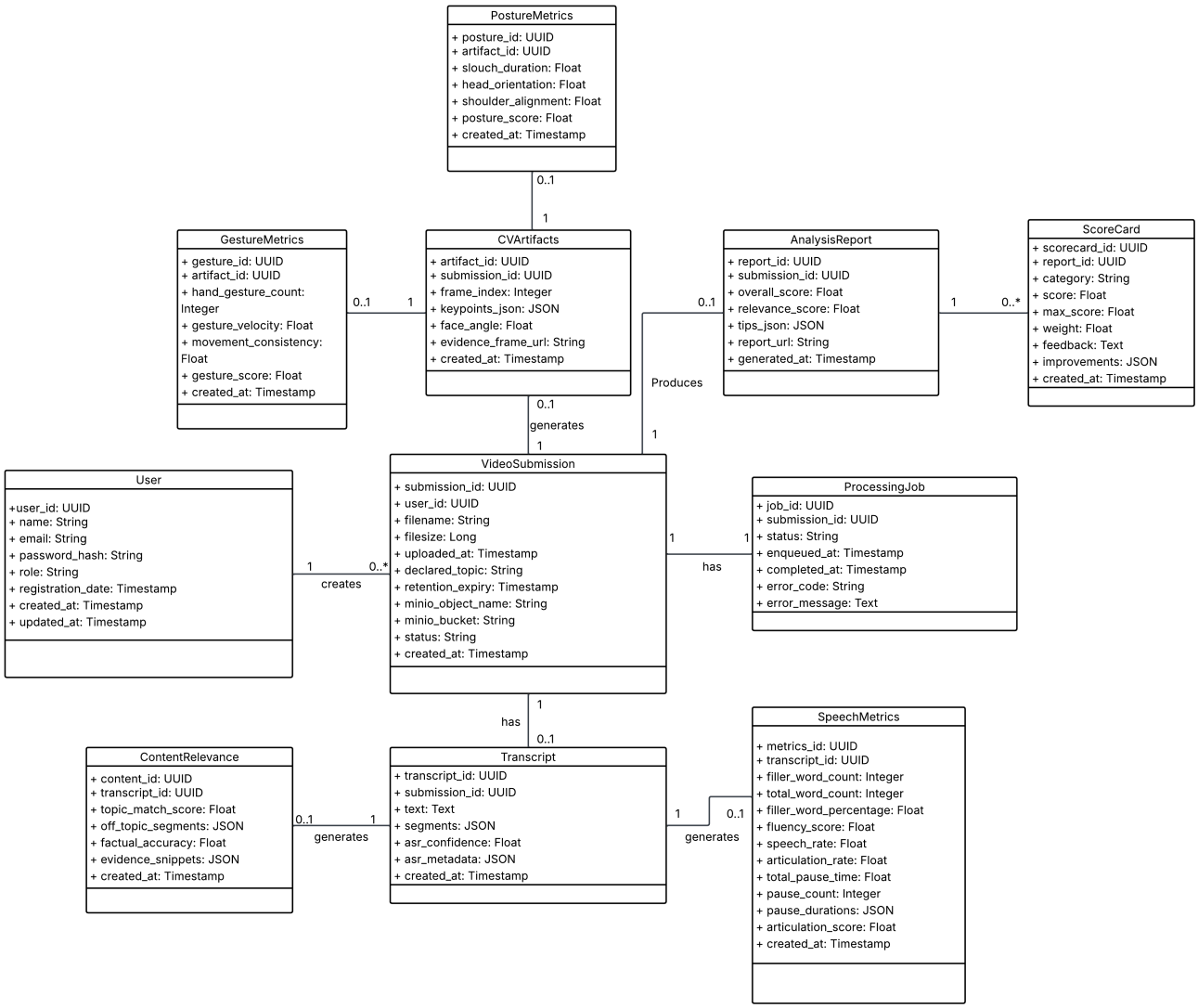

Figure 3.2: Domain Model of the OratoAI System

3.4 Design Models

3.4.1 Activity Diagram

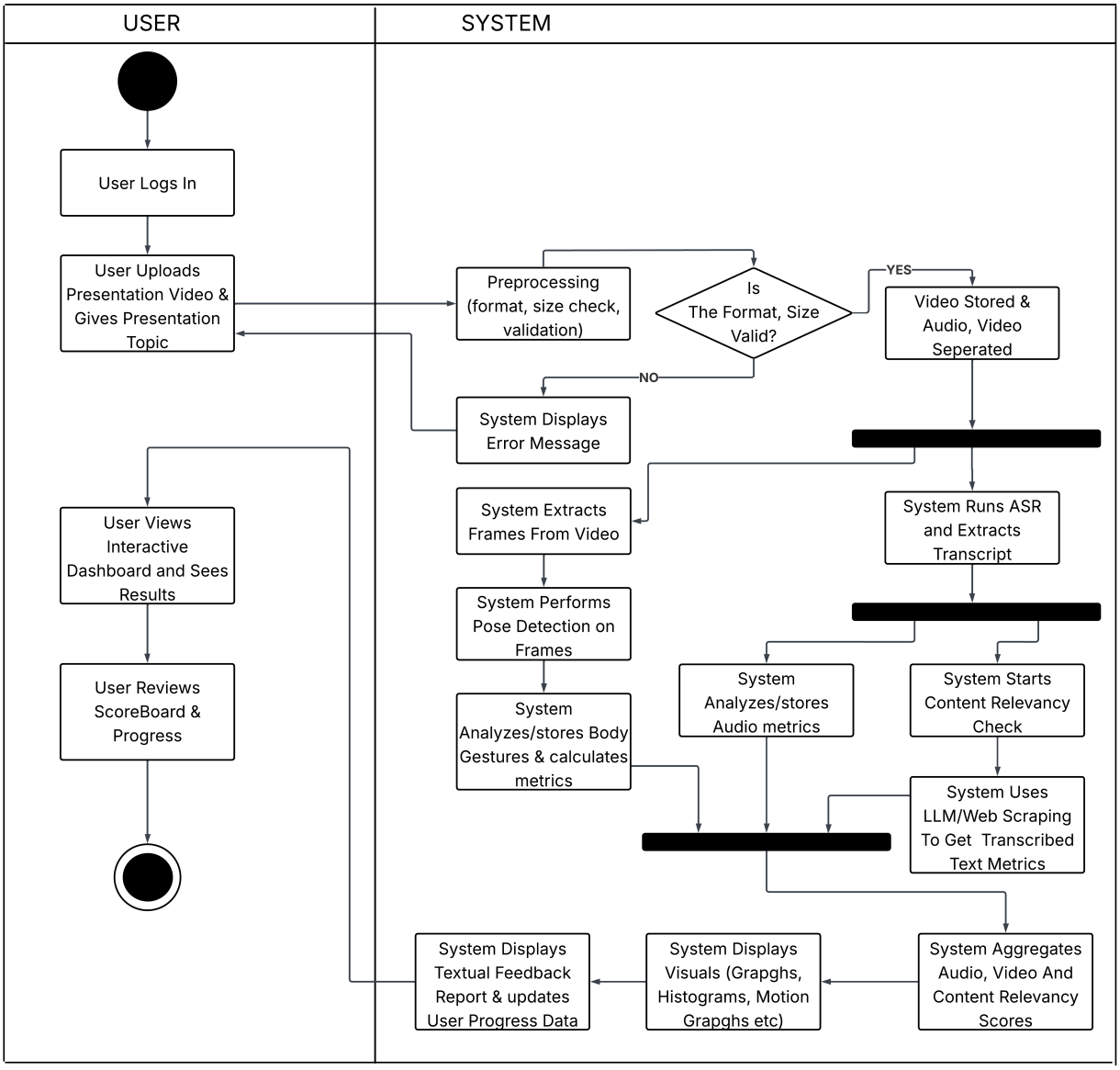

Figure 3.3: Activity Diagram of the System

3.4.2 Class Diagram

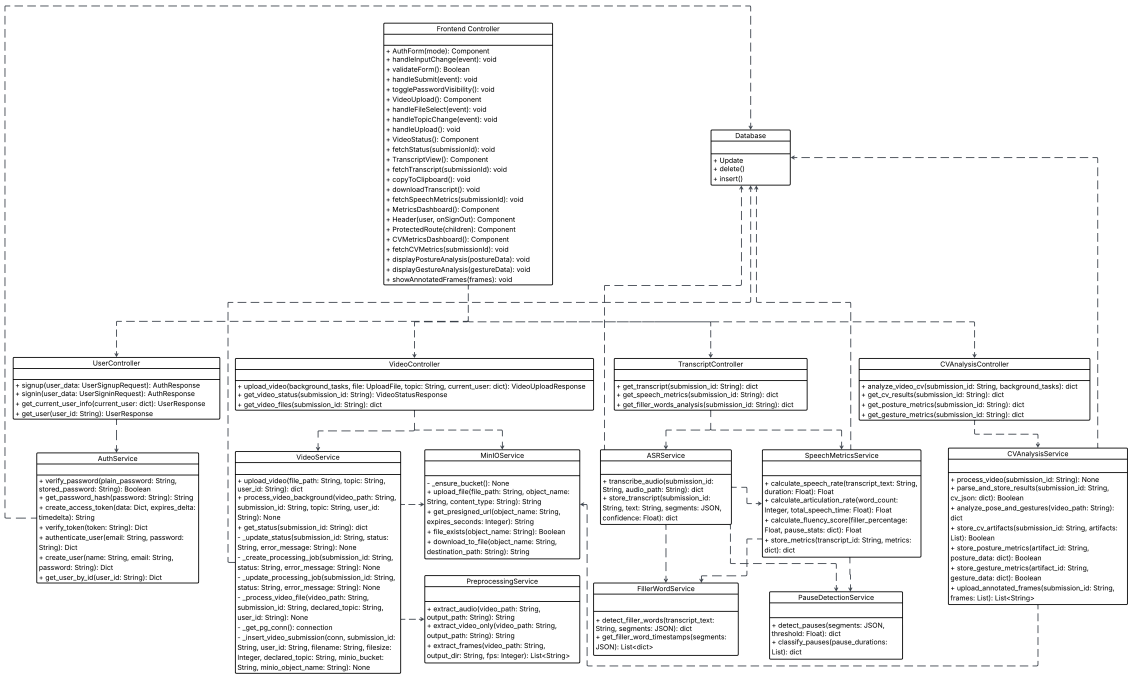

Figure 3.4: Class Diagram of the System

3.4.3 Class-level Sequence Diagrams

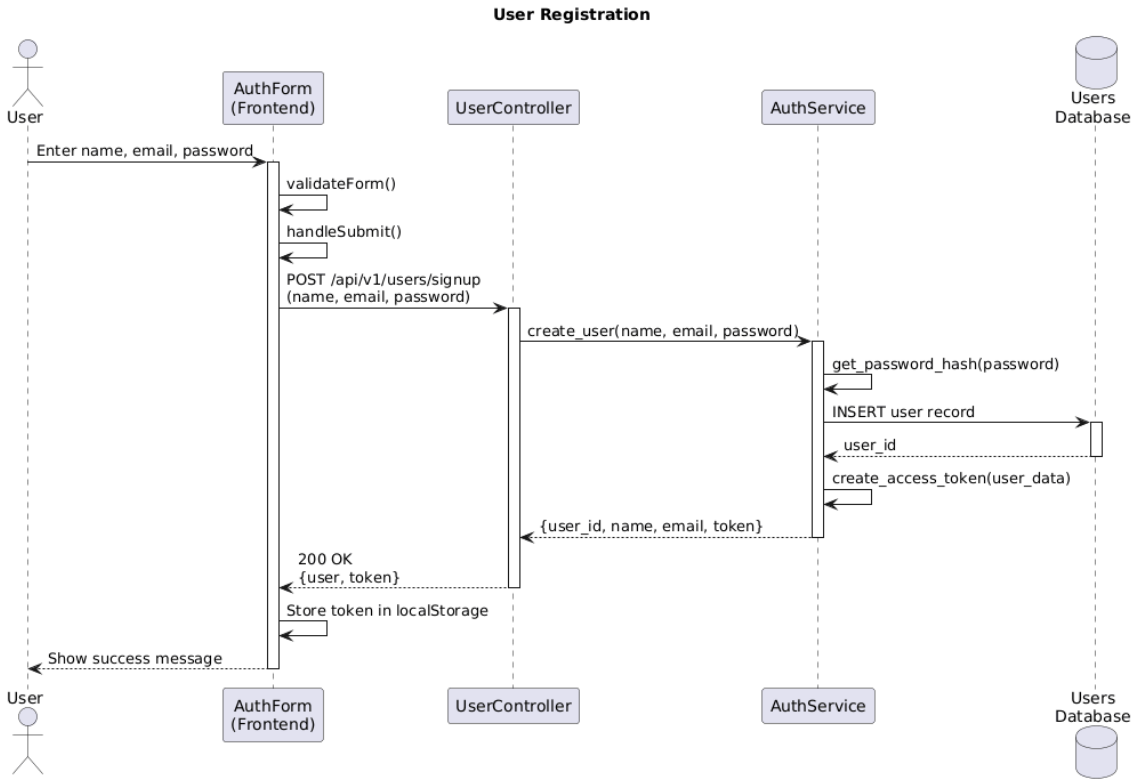

Figure 3.5: User Registration

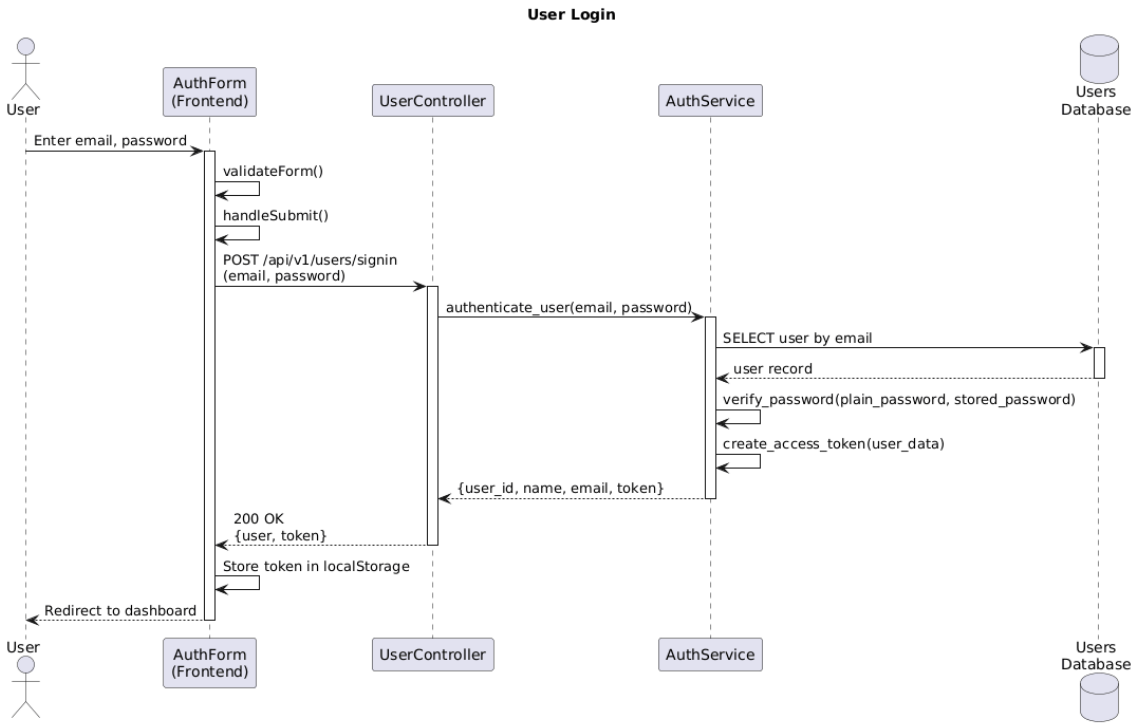

Figure 3.6: User Login

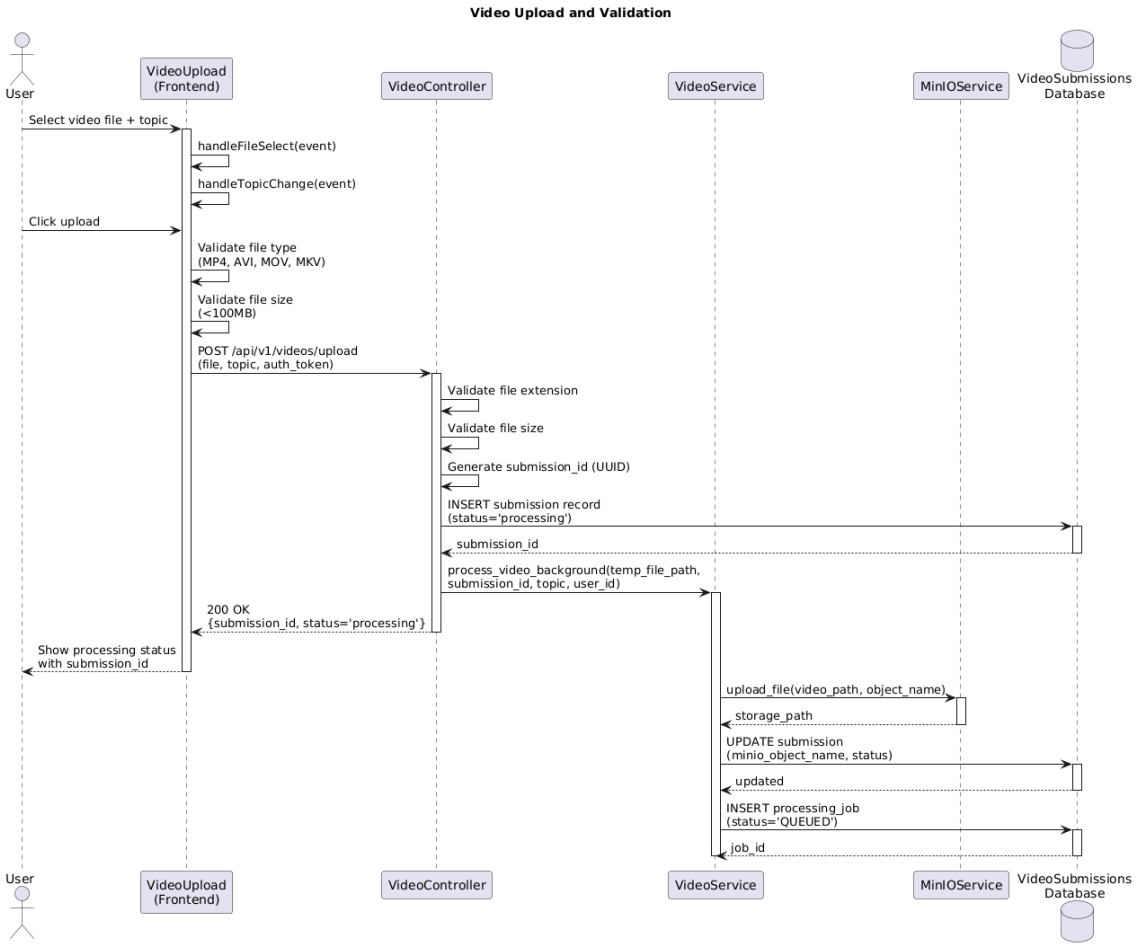

Figure 3.7: Video Upload and Validation

Figure 3.8: Video Preprocessing Workflow

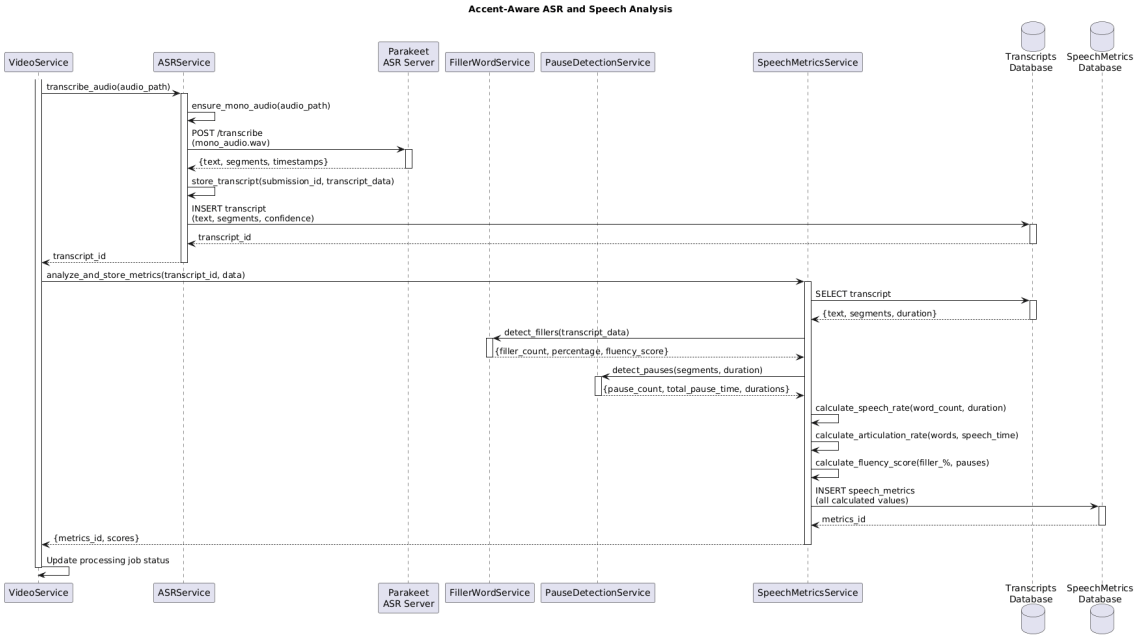

Figure 3.9: Accent-Aware Speech Recognition and Analysis

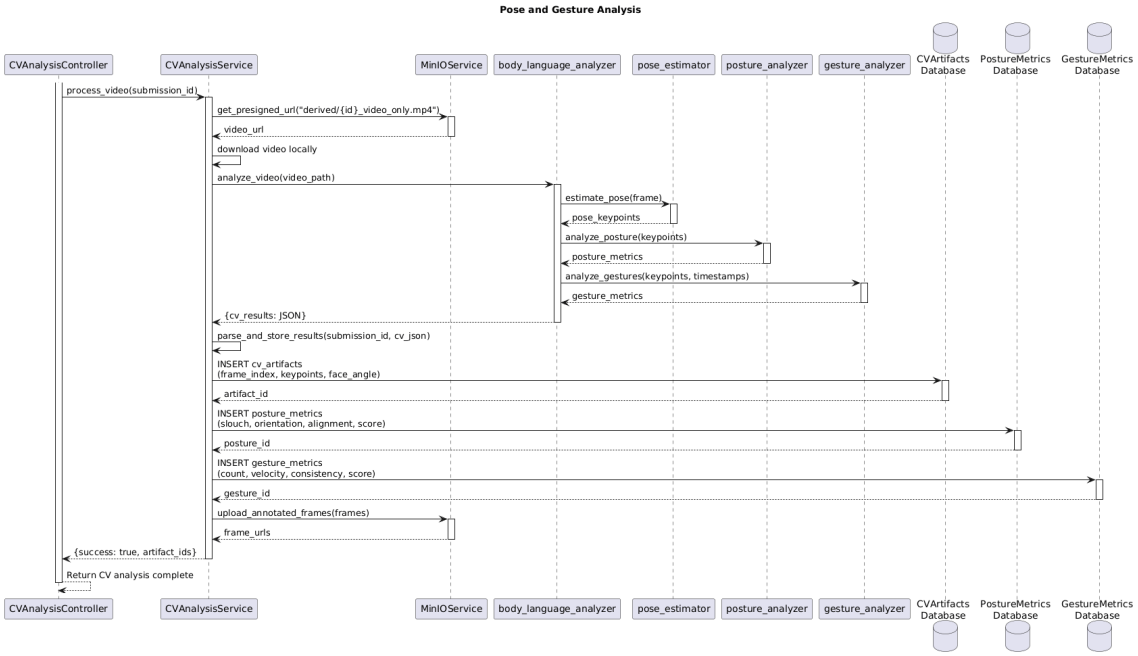

Figure 3.10: Pose and Gesture Analysis

3.4.4 System-level Sequence Diagram

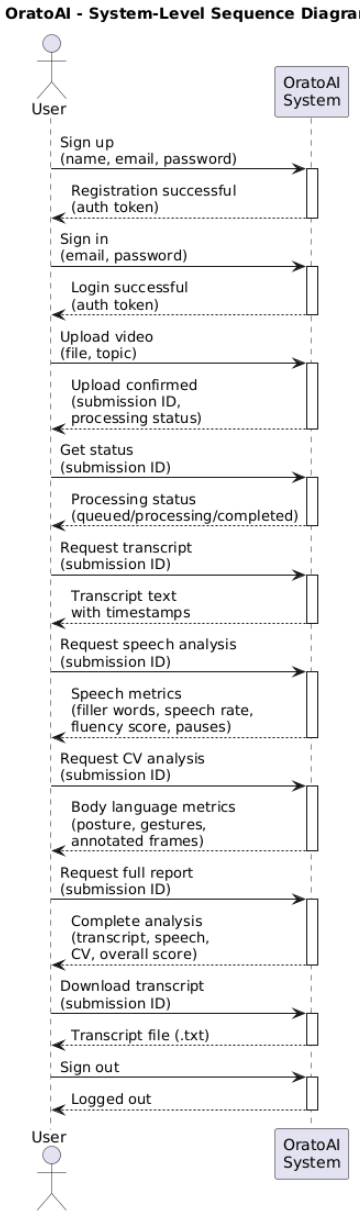

Figure 3.11: System-level Sequence Diagram

3.4.5 State Transition Diagram

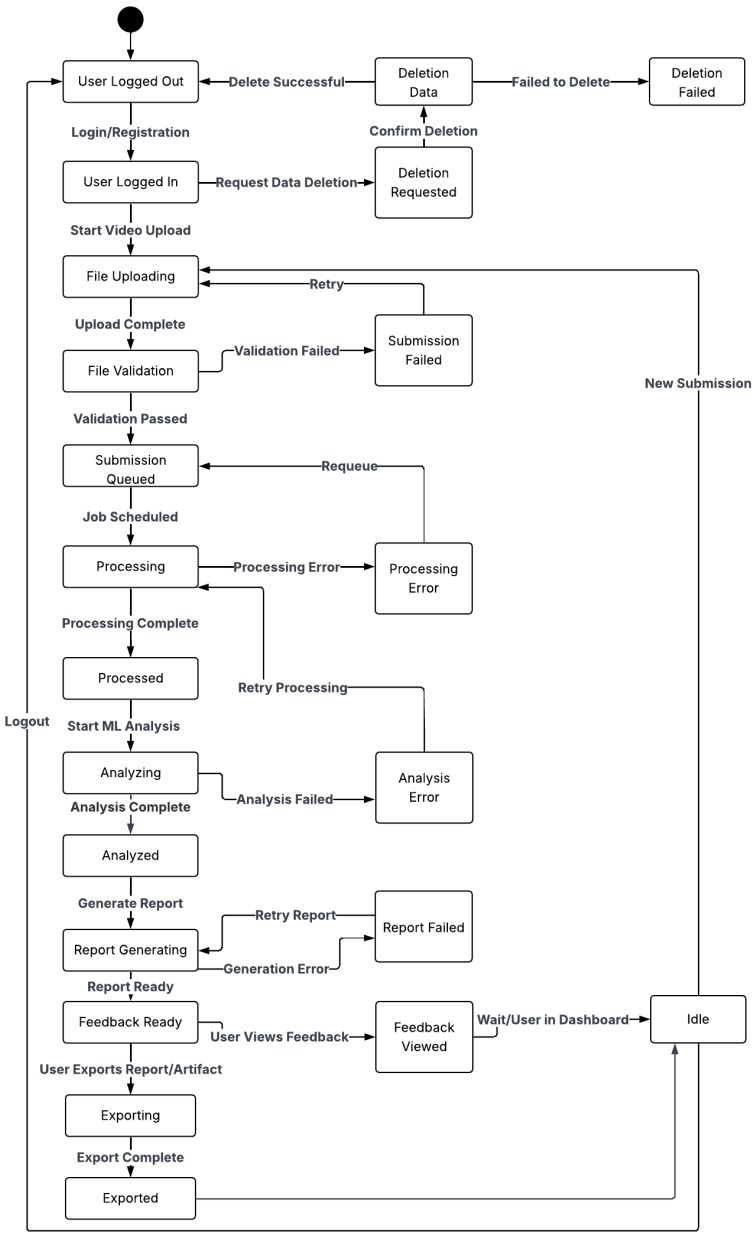

Figure 3.12: State Transition Diagram

3.4.6 Entity Relationship Diagram

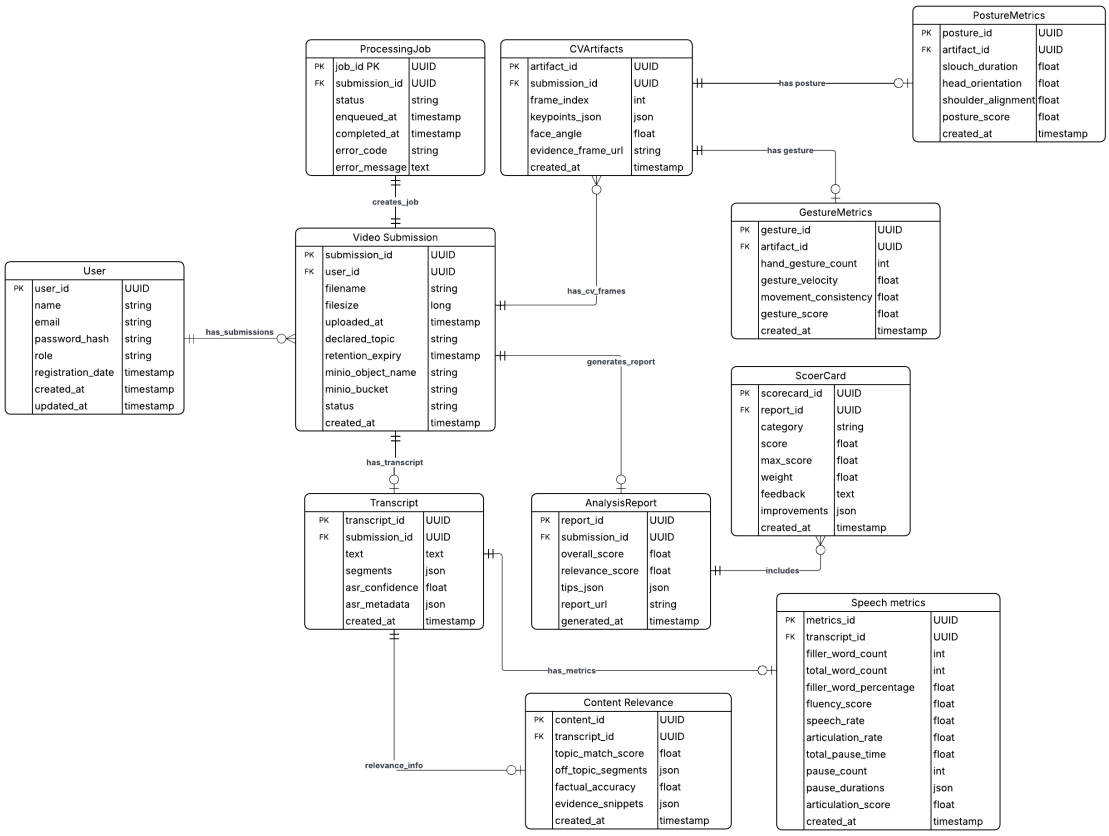

Figure 3.13: Entity Relationship Diagram

3.4.7 Data Flow Diagram (Level 0)

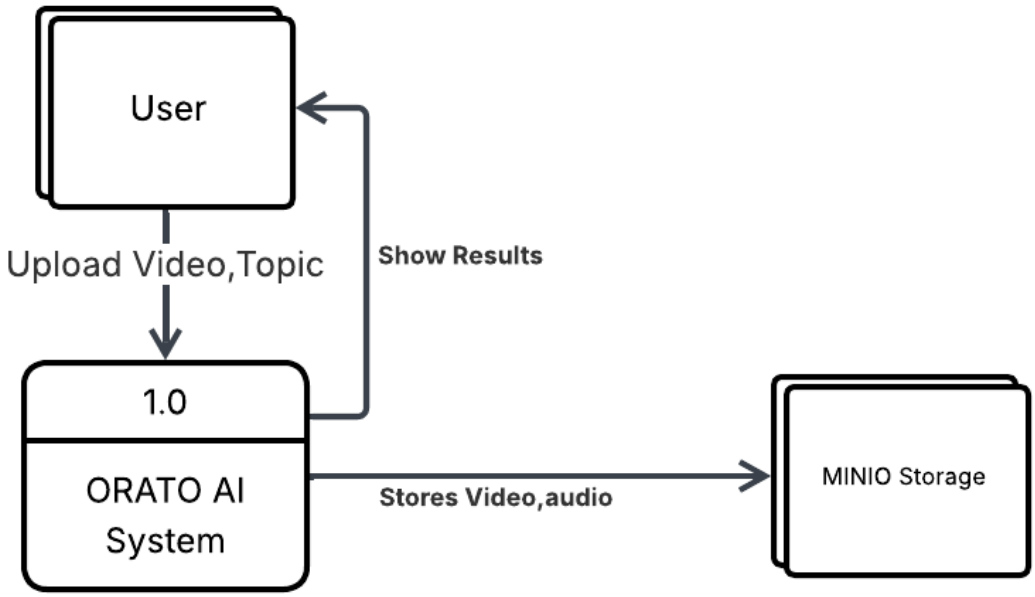

Figure 3.14: Data Flow Diagram (Level 0)

3.4.8 Data Flow Diagram (Level 1)

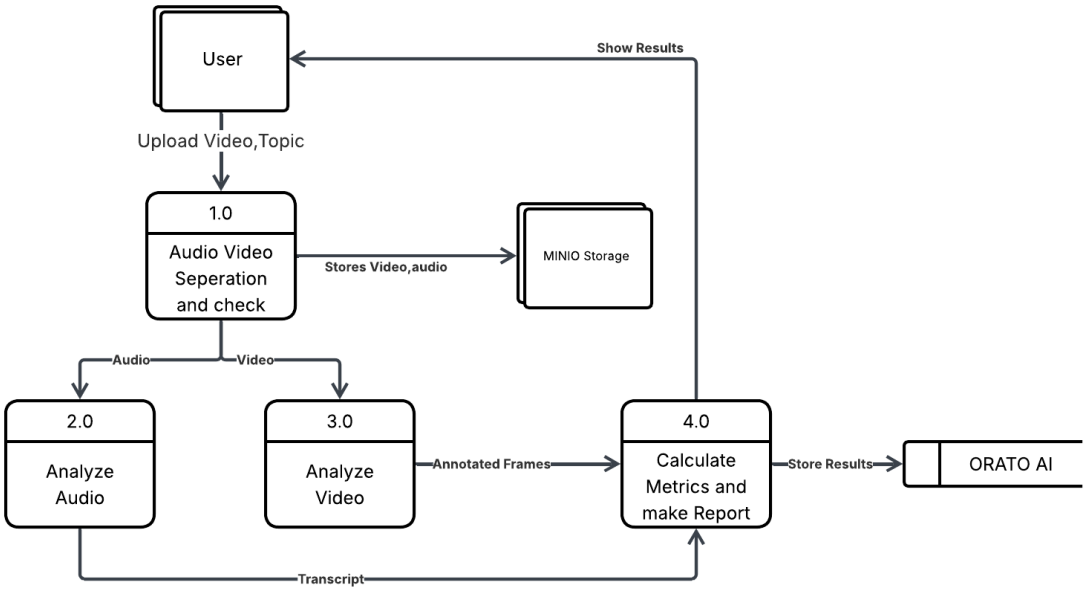

Figure 3.15: Data Flow Diagram (Level 1)

3.4.9 Data Flow Diagram (Level 2)

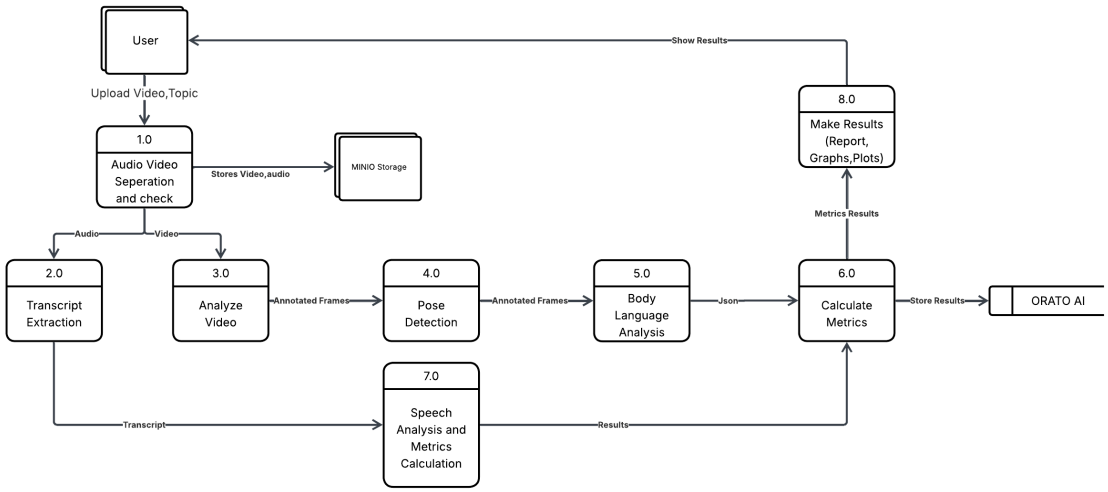

Figure 3.16: Data Flow Diagram (Level 2)

Chapter 4

Implementation and Testing

4.1 Algorithm Design

This section presents the core algorithms and pseudocode for the major modules of the OratoAI system. Each algorithm is designed to handle specific aspects of the presentation analysis pipeline.

4.1.1 User Authentication

The authentication system employs JWT (JSON Web Token) based authentication with bcrypt password hashing to ensure secure user access.

Algorithm 1: User Registration and Authentication

Input: name, email, password
Output: auth_token or error

```
/* Registration Process                                          */
if user with email exists then
    return Error: User already exists
end
/* Hash password using bcrypt                                    */
password_hash ← bcrypt.hash(password, salt_rounds = 12)
/* Generate unique user ID                                       */
user_id ← UUID.generate()
/* Store in database                                             */
INSERT INTO users (user_id, name, email, password_hash)
/* Generate JWT token                                            */
payload ← {user_id, email, exp : current_time + 24h}
auth_token ← JWT.encode(payload, secret_key)
return auth_token

/* Login Process                                                 */
Function Login(email, password):
    user ← SELECT FROM users WHERE email = email
    if user is NULL then
        return Error: Invalid credentials
    end
    is_valid ← bcrypt.verify(password, user.password_hash)
    if NOT is_valid then
        return Error: Invalid credentials
    end
    auth_token ← JWT.encode({user_id, email}, secret_key)
    return auth_token
```

The algorithm uses bcrypt for password hashing with a computational cost factor of 12, ensuring resistance against brute-force attacks.

4.1.2 Video Upload and Validation

This algorithm handles video upload with comprehensive validation checks before accepting files for processing.

Algorithm 2: Video Upload and Validation

Input: video_file, topic, user_id
Output: submission_id or error

```
/* File format validation                                    */
allowed_formats ← {.mp4, .avi, .mov, .mkv}
file_extension ← extract_extension(video_file.filename)
if file_extension ∉ allowed_formats then
    return Error: Invalid file format
end
/* File size validation (max 100MB)                          */
max_size ← 100 × 1024 × 1024 bytes
if video_file.size > max_size then
    return Error: File too large
end
/* Save to temporary location                                */
temp_path ← save_temporary(video_file)
/* Generate submission ID                                    */
submission_id ← UUID.generate()
/* Create database record                                    */
INSERT INTO video_submissions
(submission_id, user_id, filename, filesize, topic, status =′ processing′)
/* Queue background processing                               */
background_tasks.add(process_video, temp_path, submission_id)
return submission_id
```

File validation ensures only supported video formats are accepted.

4.1.3 Background Task Scheduling

The asynchronous task scheduling system enables non-blocking video processing while maintaining responsive user experience.

Algorithm 3: Background Task Scheduler

Input: video_path, submission_id, topic, user_id

Output: None (updates database status)

```
   /* Update job status to QUEUED                                    */
1  INSERT INTO processing_jobs
2  (submission_id, status =' QUEUED', enqueued_at = NOW())
   /* Start asynchronous processing                                  */
3  begin
       /* Update status to PROCESSING                                */
4      UPDATE processing_jobs SET status='PROCESSING'
       /* Execute pipeline stages                                    */
5      preprocess_video(video_path, submission_id)
6      extract_audio(video_path, submission_id)
7      transcribe_audio(submission_id)
8      analyze_speech(submission_id)
9      analyze_body_language(submission_id)
       /* Mark as completed                                          */
10     UPDATE processing_jobs SET status='DONE', completed_at=NOW()
11     UPDATE video_submissions SET status='completed'
12 end
   /* On Exception:  Handle failures                                 */
13 if processing fails then
14     UPDATE processing_jobs SET status='FAILED',
        error_message=exception.message
15     UPDATE video_submissions SET status='failed'
16 end
```

The scheduler uses try-catch error handling to ensure failures are logged.

4.1.4 Audio Extraction

Audio extraction from video files is performed using the moviepy library, splitting video into separate streams.

Algorithm 4: Video Processing and Audio Extraction

Input: video_path, submission_id

Output: video_only_path, audio_path

```
/* Create temporary directory                                    */
temp_dir ← create_temp_directory()
base_name ← extract_basename(video_path)
/* Define output paths                                           */
output_video_path ← temp_dir/base_name + "_video_only.mp4"
output_audio_path ← temp_dir/base_name + "_audio_only.wav"
/* Load video using moviepy                                      */
clip ← VideoFileClip(video_path)
/* Validate duration and audio                                   */
if clip.duration > max_duration_seconds then
    return Error: Video too long
end
if clip.audio is NULL then
    return Error: No audio track
end
/* Extract streams                                               */
clip.write_videofile(output_video_path, audio = False)
clip.audio.write_audiofile(output_audio_path)
clip.close()
/* Upload to MinIO storage                                       */
original_obj ← "uploads/" + submission_id + ".mp4"
video_obj ← "derived/" + submission_id + "_video_only.mp4"
audio_obj ← "derived/" + submission_id + "_audio_only.wav"
minio.upload_file(video_path, original_obj)
minio.upload_file(output_video_path, video_obj)
minio.upload_file(output_audio_path, audio_obj)
return (video_obj, audio_obj)
```

The algorithm creates files suitable for parallel ASR and CV processing.

4.1.5 ASR Transcription

Automatic Speech Recognition using NVIDIA Parakeet TDT 0.6B model with accent-aware capabilities.

Algorithm 5: ASR Transcription with Parakeet

Input: audio_path, submission_id
Output: transcript_id

```
/* Load audio file                                                    */
audio_data, sample_rate ← soundfile.read(audio_path)
/* Ensure mono audio                                                  */
if audio_data.ndim > 1 then
    audio_data ← np.mean(audio_data, axis = 1)          // Stereo to mono
end
/* Load Parakeet ASR model                                            */
model ←
  nemo_asr.ASRModel.from_pretrained("nvidia/parakeet-tdt-0.6b-v2")
/* Perform transcription with timestamps                              */
output ← model.transcribe([audio_path], return_hypotheses = True)
/* Extract results                                                    */
full_text ← output[0].text
word_timestamps ← output[0].timestamp['word']
segment_timestamps ← output[0].timestamp['segment']
confidence ← 0.9                                    // Default high confidence
/* Store in database                                                  */
INSERT INTO transcripts (submission_id, text = full_text, segments =
  segment_timestamps, asr_confidence = confidence, asr_metadata =
  word_timestamps)
RETURNING transcript_id INTO transcript_id
return transcript_id
```

The Parakeet model provides word-level and segment-level timestamps.

4.1.6 Filler Word Detection

Algorithm 6: Filler Word Detection with Timestamps

Input: transcript_data (full_text, word_timestamps, audio_duration)

Output: filler_count, filler_percentage, fluency_score, weighted_count

```
/* Define filler word lexicon with severity weights                    */
filler_lexicon ← {"um" : 1.5, "uh" : 1.5, "like" : 1.0, "you know" :
  1.2, "basically" : 1.0, "actually" : 1.0, "literally" : 1.2}
/* Check if timestamps available for accuracy                          */
if word_timestamps is available then
    /* Timestamp-based analysis (more accurate)                        */
    filler_count ← 0; weighted_count ← 0.0
    total_words ← |word_timestamps|; filler_details ← []
    for each word in word_timestamps do
        word_text ← word.text.lower()
        if word_text ∈ filler_lexicon then
            filler_count += 1
            weight ← filler_lexicon[word_text]
            weighted_count += weight
            filler_details.append({word : word_text, timestamp :
              word.start_time, severity : weight})
        end
    end
else
    /* Fallback:  Text-only analysis                                   */
    words ← full_text.split()
    total_words ← |words|
    filler_count ← 0; weighted_count ← 0.0
    for word in words do
        word_clean ← word.lower().strip_punctuation()
        if word_clean ∈ filler_lexicon then
            filler_count += 1
            weighted_count += filler_lexicon[word_clean]
        end
    end
end
/* Calculate metrics                                                   */
filler_percentage ← (filler_count / total_words) × 100
base_penalty ← filler_percentage × 2                          // Max 40 points
density_penalty ← min((filler_count / audio_duration) × 10, 30)   // Fillers per second
fluency_score ← max(0, 100 − base_penalty − density_penalty)
return (filler_count, filler_percentage, weighted_count, fluency_score)
```

4.1.7 Pause Detection

Algorithm 7: Pause Detection and Statistical Analysis

Input: segment_timestamps, audio_duration

Output: pauses, summary_statistics

```
/* Constants                                                    */
PAUSE_THRESHOLD ← 0.2
SHORT_MAX ← 0.5; MEDIUM_MAX ← 1.0; EXTREME_MIN ← 2.0

/* Stage 1: Calculate all gaps between segments                 */
gaps ← []
for i ← 0 to |segment_timestamps| − 2 do
    current_end ← segment_timestamps[i].end_time
    next_start ← segment_timestamps[i + 1].start_time
    gap_duration ← next_start − current_end
    if gap_duration > 0 then
        gaps.append({start : current_end, end : next_start, duration :
          gap_duration})
    end
end

/* Stage 2 & 3: Filter and Classify                             */
pauses ← []
for gap in gaps do
    if gap.duration ≥ PAUSE_THRESHOLD then
        if gap.duration < SHORT_MAX then
            gap.category ← "short"
        else if gap.duration < MEDIUM_MAX then
            gap.category ← "medium"
        else if gap.duration < EXTREME_MIN then
            gap.category ← "long"
        else
            gap.category ← "extreme"
        end
        pauses.append(gap)
    end
end

/* Stage 4: Calculate summary statistics                        */
pause_count ← |pauses|
total_pause_time ← ∑_{p∈pauses} p.duration
net_speaking_time ← audio_duration − total_pause_time
if pause_count > 0 then
    avg_pause ← total_pause_time / pause_count
    max_pause ← max_{p∈pauses} p.duration
end
avg_pause ← 0; max_pause ← 0
summary ←
```

The multi-stage pipeline calculates gaps and generates comprehensive statistics.

4.1.8 Speech Metrics Calculation

Comprehensive speech quality metrics including speech rate, articulation rate, and fluency score.

Algorithm 8: Speech Quality Metrics

Input: transcript_text, total_duration, pause_stats

Output: speech_rate, articulation_rate, fluency_score

`/* Calculate word count */`

$words \leftarrow$ transcript_text.split()

$word\_count \leftarrow |words|$

`/* Speech Rate (SR): total words per minute */`

$speech\_rate \leftarrow \frac{word\_count}{total\_duration} \times 60$

`/* Articulation Rate (AR): words per minute excluding pauses */`

$speech\_time \leftarrow total\_duration - pause\_stats.total\_pause\_time$

$articulation\_rate \leftarrow \frac{word\_count}{speech\_time} \times 60$

`/* Continuity Differential D = (AR - SR) / AR */`

$continuity\_diff \leftarrow \frac{articulation\_rate - speech\_rate}{articulation\_rate} \times 100$

`/* Categorize speech rate */`

if $speech\_rate < 120$ **then**

  $rate\_category \leftarrow$ "Slow"

else if $speech\_rate \leq 160$ **then**

  $rate\_category \leftarrow$ "Normal"

else

  $rate\_category \leftarrow$ "Fast"

end

`/* Calculate fluency score (0-100) */`

$filler\_penalty \leftarrow \min(filler\_percentage \times 2, 40)$ `// Max 40 pts`

$pause\_penalty \leftarrow \min(\frac{pause\_stats.pause\_count}{word\_count} \times 100, 30)$ `// Max 30 pts`

$fluency\_score \leftarrow \max(0, 100 - filler\_penalty - pause\_penalty)$

return $(speech\_rate, articulation\_rate, fluency\_score)$

The fluency score combines filler word frequency and pause patterns.

4.1.9 Pose Estimation

MediaPipe BlazePose extracting 33 body landmarks with 3D coordinates.

Algorithm 9: 3D Pose Extraction

Input: video_path

Output: pose_keypoints_3D

```
pose_detector ← MediaPipe.Pose(model_complexity = 1)
pose_keypoints_3D ← []
for frame in video do
    results ← pose_detector.process(frame)
    if pose detected then
        keypoints ← {x, y, z, visibility} for all 33 landmarks
        pose_keypoints_3D.append(keypoints)
    end
end
return pose_keypoints_3D
```

Each landmark includes normalized coordinates $(x, y) \in [0, 1]$ and relative depth z from hip midpoint.

4.1.10 Posture Analysis

Algorithm 10: Posture Analysis

Input: pose_keypoints_3D

Output: neck_flexion, CCA, slouch_%

$flex\_angles \leftarrow []$;
$cca\_angles \leftarrow []$;
$slouch\_frames \leftarrow 0$
for kp ***in*** $pose\_keypoints\_3D$ **do**
    $shoulder\_mid \leftarrow (kp.L\_shoulder + kp.R\_shoulder)/2$
    $hip\_mid \leftarrow (kp.L\_hip + kp.R\_hip)/2$
    $C7 \leftarrow \{shoulder\_mid.x,\ shoulder\_mid.y + 0.03,\ shoulder\_mid.z\}$
    `/* 3D Neck Flexion */`
    $\vec{neck} \leftarrow [kp.nose - C7]$
    $\theta \leftarrow angle(\vec{neck},\ \vec{vertical})$
    $flex\_angles.append(|90^\circ - \theta|)$
    `/* 2D Craniocervical Angle */`
    $\alpha \leftarrow \arctan\left(\frac{kp.ear.y - C7.y}{kp.ear.x - C7.x}\right)$
    $cca\_angles.append(180^\circ - |\alpha|)$
    if $|90^\circ - \theta| > 20^\circ$ **then**
        $slouch\_frames \mathrel{+}= 1$
    end
end
return $(mean(flex\_angles),\ mean(cca\_angles),\ slouch\_frames/total \times 100)$

Note: 3D flexion captures multi-directional neck deviation; CCA uses 2D due to C7 estimation constraints.

4.1.11 Gesture Analysis

3D wrist tracking with peak-based gesture detection and Normalized Jerk Cost.

Algorithm 11: Gesture Frequency & Smoothness

Input: pose_keypoints_3D, fps

Output: GPM, mean_NJC

`/* Extract wrists in 3D */`

$wrist\_3D \leftarrow [kp.L\_wrist,\ kp.R\_wrist]$ for all frames

`/* 3D velocity magnitude */`

$vel \leftarrow \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2}/\Delta t$

`/* Gesture peak detection */`

$peaks \leftarrow find\_peaks(vel,\ height = mean(vel) \times 4)$

$GPM \leftarrow \frac{|peaks|}{duration} \times 60$

`/* Normalized Jerk Cost (Flash & Hogan 1985) */`

$njc\_values \leftarrow []$

for *segment* ***in*** *gesture_segments* **do**

  $jerk \leftarrow \frac{d^3\vec{p}}{dt^3}$

  $NJC_{raw} \leftarrow \sqrt{\frac{T^5}{2L^2} \int jerk^2\, dt}$

  $njc\_values.append(NJC_{raw}/1100)$

end

return $(GPM,\ mean(njc\_values))$

Interpretation: NJC < 0.4 = smooth, 0.4–0.7 = moderate, > 0.7 = jerky. Target gesture frequency: 10–16 GPM.

4.1.12 Head Pose Estimation

3D head orientation (yaw, pitch, roll) via Perspective-n-Point.

Algorithm 12: Head Pose via solvePnP

Input: video_frames

Output: yaw, pitch, roll

$face\_mesh \leftarrow$ MediaPipe.FaceMesh()

`/* 3D facial keypoint model (cm) */`

$model\_3D \leftarrow [(0,0,0),\ (0,-330,-65),\ (-225,170,-135),\ ...]$

`/* Camera intrinsic matrix */`

$$K \leftarrow \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

for $frame$ ***in*** $video\_frames$ **do**

  $landmarks\_2D \leftarrow face\_mesh.process(frame)$

  if *face detected* **then**

    $(R,\ T) \leftarrow solvePnP(model\_3D,\ landmarks\_2D,\ K)$

    `/* Euler angle extraction */`

    $yaw \leftarrow \arctan 2(R_{1,0},\ R_{0,0})$

    $pitch \leftarrow \arctan 2(-R_{2,0},\ \sqrt{R_{0,0}^2 + R_{1,0}^2})$

    $roll \leftarrow \arctan 2(R_{2,1},\ R_{2,2})$

  end

end

return $(yaw,\ pitch,\ roll)$ in degrees

Eye Contact Rule: facing camera if $|yaw| < 30^\circ$ and $|pitch| < 20^\circ$.

Angle Meaning: Yaw = left/right, Pitch = up/down, Roll = tilt.

4.2 External APIs/SDKs

API/SDK & Version	Description	Purpose of Usage	API Endpoint/Function Used
NVIDIA NeMo Toolkit (v1.22.0)	End-to-end deep learning framework with Parakeet TDT 0.6B ASR model	Accent-aware automatic speech recognition	`nemo_asr.ASRModel.from_pretrained()`, `model.transcribe()`
MediaPipe (v0.10.9)	Google's cross-platform ML solutions for media	Pose estimation, hand landmark detection	`mediapipe.solutions.pose.Pose()`, `pose.process()`
MinIO Python SDK (v7.2.0)	High-performance S3-compatible object storage	Video/audio file storage management	`Minio()`, `put_object()`, `get_presigned_url()`
bcrypt (v4.1.2)	Password hashing library	Secure password storage	`bcrypt.hashpw()`, `bcrypt.checkpw()`
PyJWT (v2.8.0)	JSON Web Token implementation	User session authentication	`jwt.encode()`, `jwt.decode()`
moviepy (v1.0.3)	Video editing library with FFmpeg	Video/audio stream separation	`VideoFileClip()`, `clip.audio.write_audiofile()`
psycopg2-binary (v2.9.9)	PostgreSQL database adapter	Database connection and queries	`psycopg2.connect()`, `cursor.execute()`
soundfile (v0.12.1)	Sound file reading/writing library	Audio file I/O for ASR	`soundfile.read()`, `soundfile.write()`
NumPy (v1.26.2)	Scientific computing package	Array operations, signal processing	`np.mean()`, `np.array()`, `np.arctan()`
FastAPI (v0.109.0)	Modern Python web framework	RESTful API endpoints	`@app.get()`, `@app.post()`, `BackgroundTasks`
React (v18.2.0)	JavaScript UI library	Frontend components	`useState()`, `useEffect()`, `useContext()`
Axios (v1.6.2)	Promise-based HTTP client	API requests from frontend	`axios.get()`, `axios.post()`, `axios.create()`
React Router DOM (v6.20.1)	Declarative routing for React	Client-side navigation	`BrowserRouter`, `Routes`, `Route`

Table 4.1: External APIs and SDKs used in OratoAI implementation

4.2.1 Key Integration Details

4.2.1.1 NVIDIA Parakeet ASR

The Parakeet TDT (Transducer with Dynamic Time warping) 0.6B model is specifically chosen for its:

- Accent-aware transcription capabilities
- Word-level and segment-level timestamp generation
- High accuracy on diverse English accents
- Optimized inference speed suitable for production deployment

4.2.1.2 MediaPipe Holistic

MediaPipe provides real-time body pose estimation with:

- 33 pose landmarks covering full body
- 21 hand landmarks per hand for gesture analysis
- Sub-pixel accuracy and high frame rate processing
- Cross-platform compatibility (CPU/GPU)

4.2.1.3 MinIO Object Storage

MinIO serves as the primary storage backend offering:

- S3-compatible API for easy migration
- High-performance distributed storage
- Bucket-based organization (uploads, derived media)
- Presigned URL generation for secure video access

4.3 Testing Details

4.3.1 Unit Testing

Each unit test is designed to test a specific module independently from other components.

ID	Objective	Precond	Steps	Data	Expected	Post-cond	Actual	Sts
TC01	Test Auth Module: registration & login	User not registered	1. Register user 2. Login with creds 3. Verify JWT	Name: John Doe Email: john@ex.com Pass: Pass123!	User registered, JWT returned on login, session active	User can access system	As Exp.	Pass
TC02	Test Video Upload: validation & processing	User logged in	1. Upload valid MP4 2. Upload invalid format 3. Upload large file	Valid: pres.mp4 (50MB) Invalid: doc.txt Large: vid.mp4 (150MB)	Valid accepted, invalid rejected, size error shown	Valid video in DB & MinIO	As Exp.	Pass
TC03	Test ASR Module: transcription	Audio extracted from video	1. Send audio to Parakeet 2. Receive transcript 3. Store in DB	30s speech audio (WAV)	Transcript with word/segment timestamps, confidence >0.8	Transcript saved in DB	As Exp.	Pass
TC04	Test Speech Analysis: filler, pause, rate	Transcript available	1. Detect filler words 2. Detect pauses 3. Calculate speech rate	Transcript: 150 words, 60s, "um, uh" present	Filler count, pause stats, SR=150 WPM, AR=180 WPM, fluency score calculated	Metrics in speech_metrics table	As Exp.	Pass
TC05	Test CV Module: pose & posture analysis	Video available in MinIO	1. MediaPipe pose detection 2. Analyze posture 3. Calculate scores	10s presentation video	33 landmarks/frame detected, slouch % calculated, posture score generated	CV artifacts & metrics stored	As Exp.	Pass
TC06	Test Dashboard: results display	Analysis completed	1. Navigate to dashboard 2. Select submission 3. View results	submission_id with status='completed'	Transcript, speech metrics, CV metrics displayed with charts	User can view all results	As Exp.	Pass

Table 4.2: Unit Testing for OratoAI System (Module Level)

4.3.2 Test Coverage Summary

The unit testing covers the following modules:

1. **Authentication Module** (TC01): User registration and login functionality
2. **Video Upload Module** (TC02): File format validation, size validation, and upload processing
3. **ASR Module** (TC03): Parakeet transcription accuracy and timestamp generation
4. **Speech Analysis Module** (TC04): Filler word detection, pause detection, speech rate calculation
5. **CV Analysis Module** (TC05): MediaPipe pose estimation and posture analysis
6. **Dashboard Module** (TC06): Results display and user interface

4.3.3 TC01-Registration

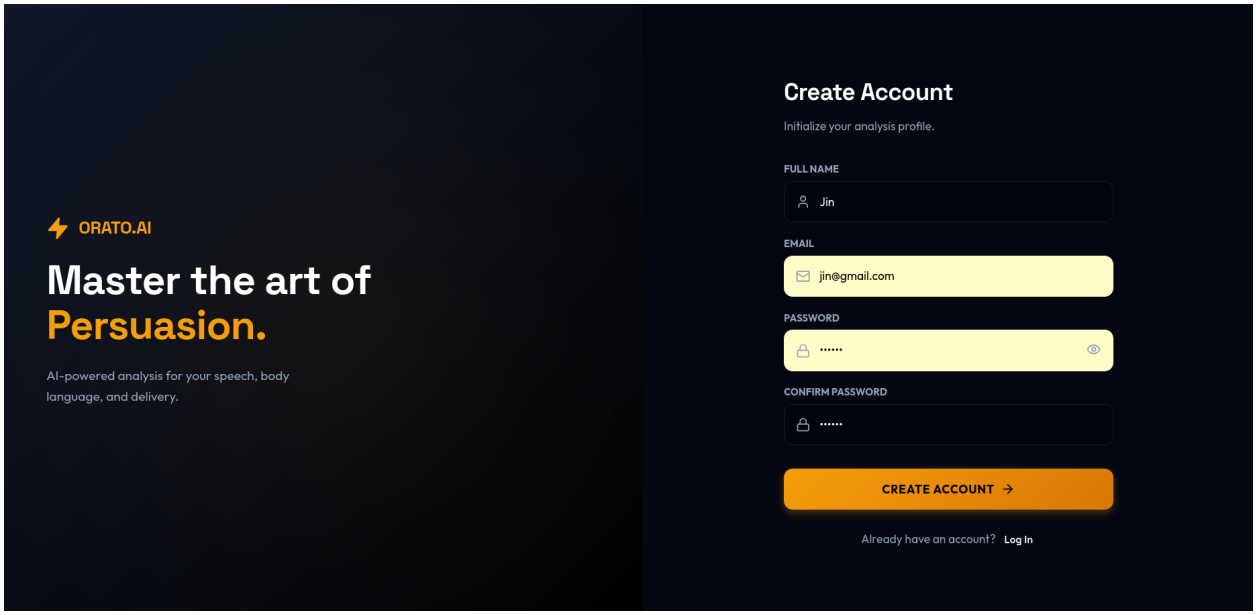

Figure 4.1: TC01-Registration

4.3.4 TC01-Login

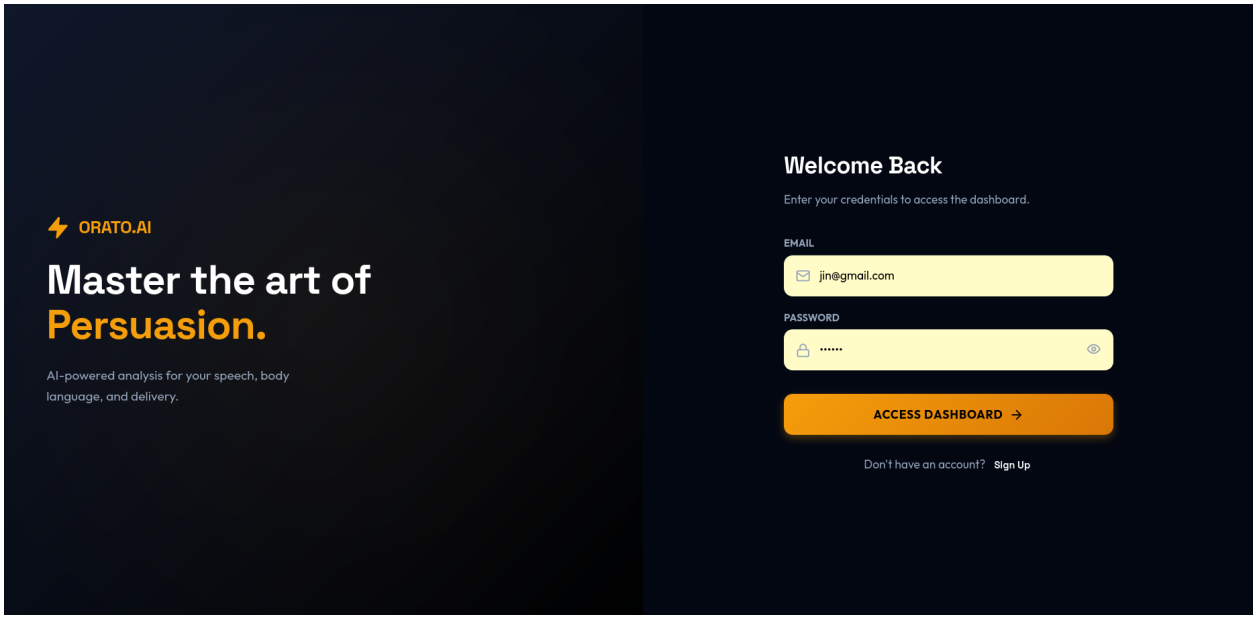

Figure 4.2: TC01-Login

4.3.5 TC02-Upload

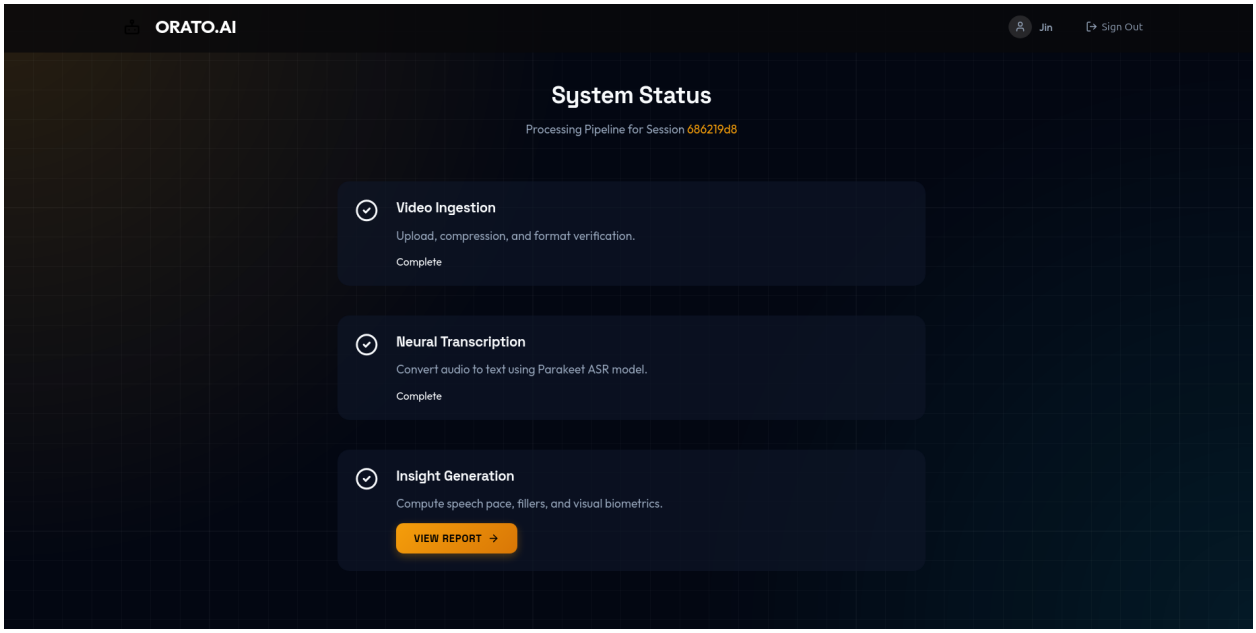

Figure 4.3: TC02-Upload

4.3.6 TC03-ASR

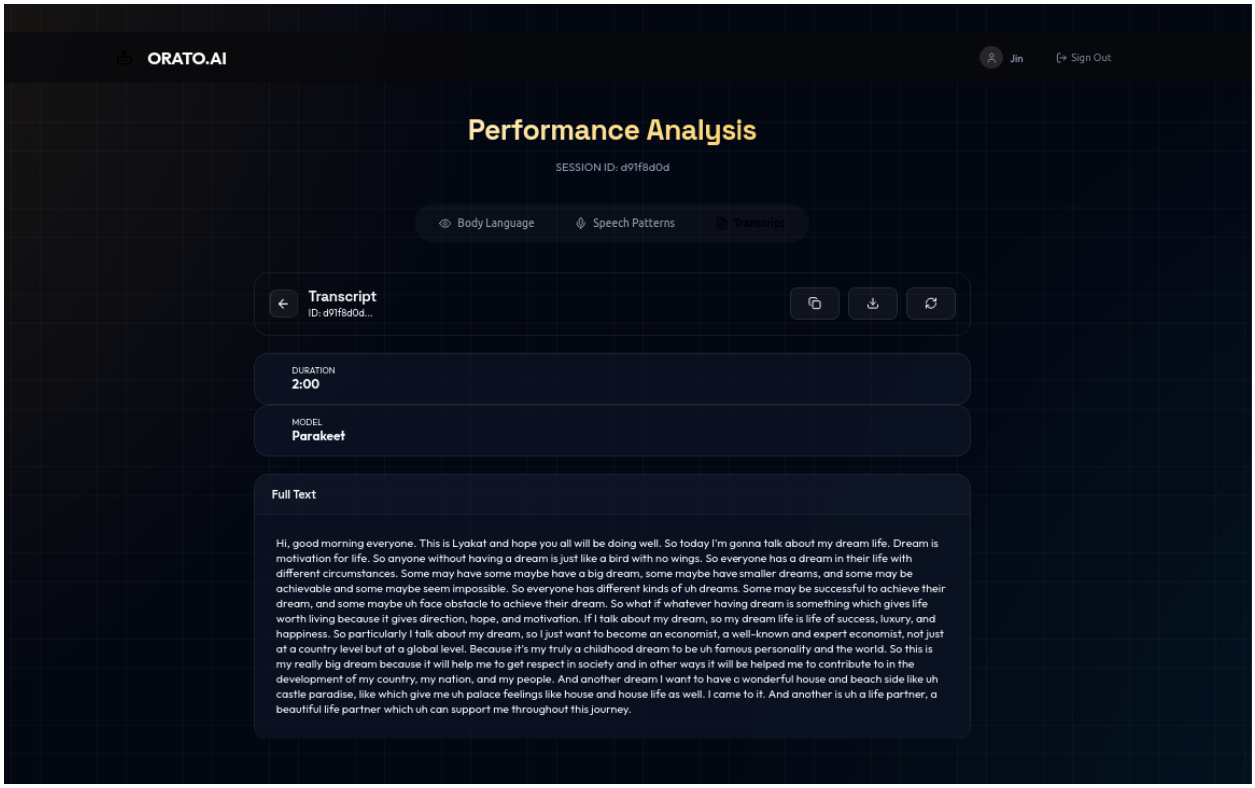

Figure 4.4: TC03-ASR

4.3.7 TC04 and TC06-Speech Analysis and Dashboard

Figure 4.5: TC04 and TC06-Speech Analysis and Dashboard

4.3.8 TC05 and TC06-CV Analysis and Dashboard

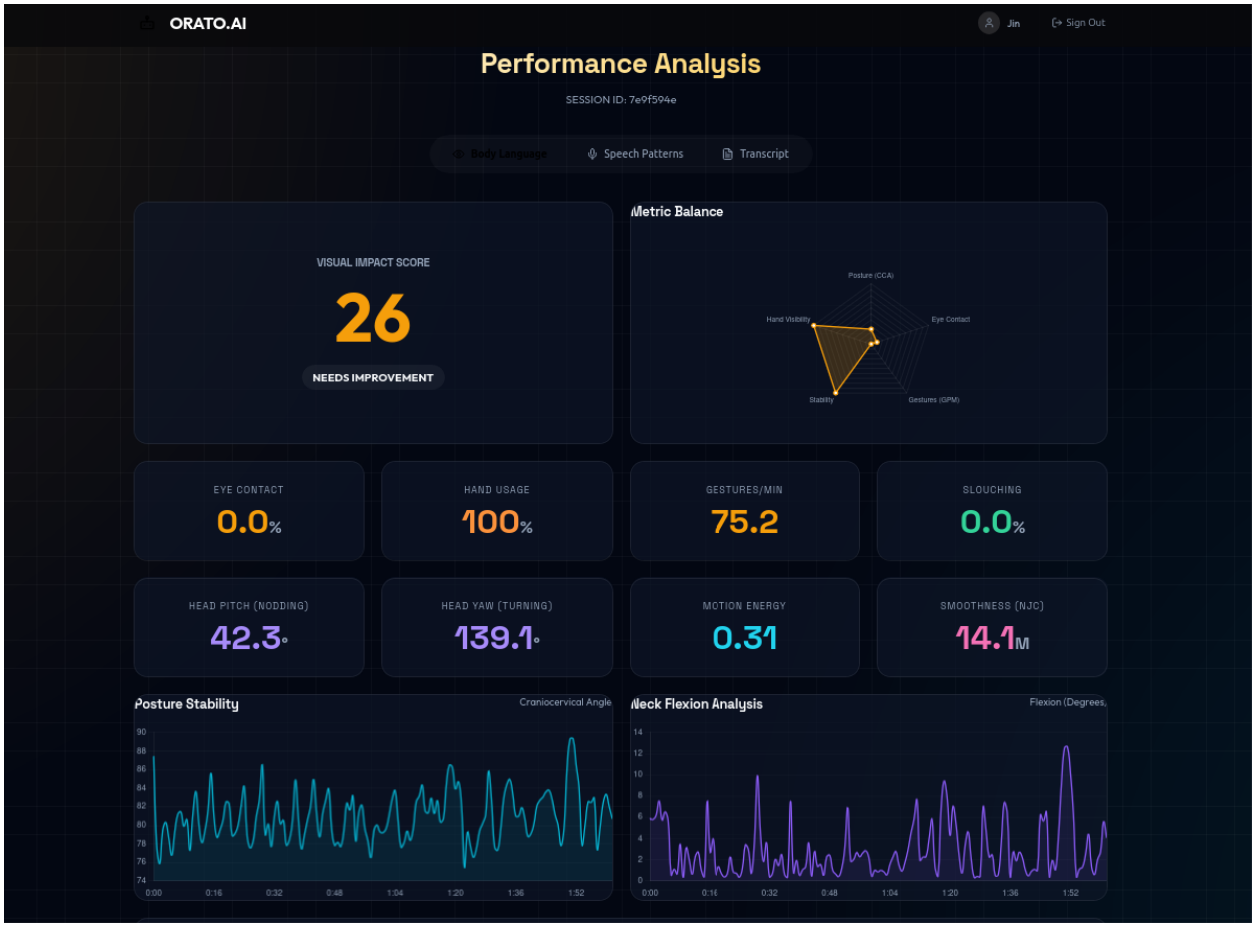

Figure 4.6: TC05 and TC06-CV Analysis and Dashboard

Bibliography

[1] Reading A-Z. Fluency standards table. https://www.readinga-z.com/fluency/standards-tabl 2023. Standards for reading fluency.

[2] ArXiv. Automated assessment of public speaking skills using multimodal cues. https://arxiv.org/pdf/2105.02636, 2021. Research #1. Source for weighted scoring matrix and PSCR alignment.

[3] ArXiv. Arxiv paper. https://arxiv.org/pdf/2301.10761, 2023. Preprint paper on ArXiv.

[4] ArXiv. Pianomotion10m: Dataset and benchmark for hand motion generation. https://arxiv.org/html/2406.09326v1, 2024. Research #41. Source for 80% hand visibility threshold.

[5] Argyle & Cook. Eye contact rules: The 50/70 rule. https://www.americanexpress.com/en-us/business/trends-and-insights/articles/the 1976. Research #34. Source for the 50/70 eye contact rule.

[6] DTU54DL. Common accent dataset – pakistan & south asia (hugging face). https://huggingface.co/datasets/DTU54DL/common-accent, 2022. A diverse speech dataset covering accents from Pakistan and South Asia, suited for accent-aware ASR research and model training.

[7] Binetti et al. (BPS). Psychologists have identified the length of eye contact people find most comfortable. https://www.bps.org.uk/research-digest/psychologists-have-identified-length-eye n.d. Research #35. Source for optimal gaze duration (3.0-3.3 seconds).

[8] T. Flash and N. Hogan. The coordination of arm movements: An experimentally confirmed mathematical model. https://www.jneurosci.org/content/jneuro/5/7/1688.full.pdf, 1985. Research #56. Foundational paper for Minimum Jerk Model and NJC.

[9] JALT. Jalt publication pdf. https://jalt-publications.org/sites/default/files/pdf-arti 2024. PDF article from JALT.

[10] MDPI. Tracking a driver's face against extreme head poses using hidden markov model. https://www.mdpi.com/2076-3417/6/5/137, 2016. Research #30. Source for head pose estimation methods.

[11] MDPI. Mapping of the upper limb work-space: Benchmarking four wrist smoothness metrics. https://www.mdpi.com/2076-3417/12/24/12643, 2022. Research #57. Source for NJC formula and comparison methodology.

[12] MushanW. Globe dataset – pakistan & south asia (hugging face). https://huggingface.co/datasets/MushanW/GLOBE, 2023. A multi-accented dataset focusing on oral proficiency, including Pakistani English and South Asian regional variations; useful for ASR evaluation and research.

[13] NCBI. Pmc article. https://pmc.ncbi.nlm.nih.gov/articles/PMC8925352/, 2023. PubMed Central article.

[14] PMC. Effect of alternative video displays on postures during microsurgery skill tasks. https://pmc.ncbi.nlm.nih.gov/articles/PMC5737936/, 2017. Research #18. Source for Neck Flexion formula and thresholds.

[15] F. Ramseyer and W. Tschacher. Motion energy analysis (mea): A primer on the assessment of motion from video. https://www.researchgate.net/publication/342664554, 2011. Research #63. Source for MEA frame differencing algorithm.

[16] ResearchGate. Head pose tracking and focus of attention recognition algorithms in meeting rooms. https://www.researchgate.net/publication/221545794, 2006. Research #29. Source for solvePnP algorithm and head orientation.

[17] ResearchGate. The assessment of body sway and the choice of stability parameter(s). https://www.researchgate.net/publication/51367375, 2011. Research #23. Source for Sway velocity formula.

[18] ResearchGate. Effects of different types of hand gestures in persuasive speech on receivers' evaluations. https://www.researchgate.net/publication/233376853, 2012. Research #47. Source for effectiveness of medium intensity gestures.

[19] ResearchGate. The effects of hand gestures on psychosocial perception: Preliminary study. https://www.researchgate.net/publication/286516732, 2015. Research #48. Source for psychosocial perception of gestures.

[20] ResearchGate. Posture during the use of electronic devices in people with chronic neck pain: A 3d motion analysis project.

https://www.researchgate.net/publication/344255101, 2020. Research #16. Source for Craniocervical Angle (CCA) thresholds.

[21] ResearchGate. The biomechanics of public speaking: Enhancing political communication through posture and gesture. https://www.researchgate.net/publication/385814266, 2024. Research #46. Source for Gesture Frequency (GPM) targets.

[22] ResearchGate. Quantifying sitting posture: Posture lab using manikin model. https://www.researchgate.net/publication/392723680, n.d. Research #17. Source for CCA calculations and metrics.

[23] Max Planck Society. Mpg pure item. https://pure.mpg.de/rest/items/item_1900232_6/compo 2023. Research item from Max Planck.

[24] Rachael Tatman. Speech accent archive dataset (kaggle). https://www.kaggle.com/datasets/rtatman/speech-accent-archive, 2020. A globally curated speech corpus including Pakistani English accent samples for accent classification and speech recognition applications.

[25] VirtualSpeech. Average speaking rate words per minute. https://virtualspeech.com/blog/average-speaking-rate-words-per-minute, 2023. Information on average speaking rates.